

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

DIPLOMOVÁ PRÁCE

Webový nástroj pro vizualizaci konceptuálního svazu



2026

Vedoucí práce:
RNDr. Martin Trnečka, Ph.D.

Bc. Radek Vymětalík

Studijní program: Aplikovaná informatika,
Specializace: Vývoj software

Bibliografické údaje

Autor: Bc. Radek Vymětalík
Název práce: Webový nástroj pro vizualizaci konceptuálního svazu
Typ práce: diplomová práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Aplikovaná informatika, Specializace: Vývoj software
Vedoucí práce: RNDr. Martin Trnečka, Ph.D.
Počet stran: 47
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: Bc. Radek Vymětalík
Title: Web tool for concept lattice visualization
Thesis type: master thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Applied Computer Science, Specialization: Software Development
Supervisor: RNDr. Martin Trnečka, Ph.D.
Page count: 47
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Diplomová práce představuje nový webový nástroj pro vizualizaci a analýzu konceptuálních svazů. Hlavním cílem bylo vytvořit uživatelsky přívětivé řešení, které zpřístupní metody formální konceptuální analýzy (FCA) širší škále uživatelů. Výsledný nástroj nabízí interaktivní vizualizaci konceptuálního svazu formou Hasseova diagramu a možnost exportu daného diagramu do rastrových nebo vektorových formátů. Přínosem nástroje je také jeho vysoký výkon, který umožňuje práci i s rozsáhlými svazy.

Synopsis

The master's thesis presents a new web tool for visualization and analysis of concept lattices. The main goal was to create a user-friendly solution that will make the methods of formal concept analysis (FCA) accessible to a wider range of users. The resulting tool offers interactive visualization of a concept lattice in the form of a Hasse diagram, along with the option to export the diagram into raster or vector formats. A key benefit of the tool is its high performance, which allows efficient work even with large lattices.

Klíčová slova: FCA; Hasseův diagram; vizualizace dat; webová aplikace

Keywords: FCA; Hasse diagram; data visualization; web application

Děkuji vedoucímu diplomové práce RNDr. Martinu Trnečkovi, Ph.D. za jeho odborné vedení, čas a poskytnuté rady.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	8
2	Základy formální konceptuální analýzy	9
2.1	Formální koncept	9
2.2	Konceptuální svaz	9
3	Existující řešení	11
3.1	GUI aplikace	11
3.2	Knihovny	12
3.3	Nedostatky uvedených řešení	14
4	Požadavky na nástroj	15
5	Použité algoritmy	16
5.1	Sugiyama framework	16
5.1.1	Rozdělení vrcholů do vrstev	16
5.1.2	Minimalizace křížení hran	17
5.1.3	Určení horizontálních souřadnic vrcholů	18
5.2	Freeseův algoritmus	19
5.3	ReDraw	19
5.4	Srovnání metod pro kreslení Hasseových diagramů	20
6	Architektura aplikace	21
6.1	Základní technologie	22
6.2	Uživatelské rozhraní	22
6.2.1	Vizuální podoba uživatelského rozhraní	23
6.3	Služby	23
6.4	Paralelní zpracování	23
6.4.1	Přerušování výpočtu	24
6.4.2	Optimalizace přenosu dat	25
6.5	Správa stavu	26
6.5.1	With-transformace	27
6.6	Datová reprezentace	27
6.6.1	Formální kontext	28
6.6.2	Formální koncepty	28
6.6.3	Konceptuální svaz	29
6.6.4	Rozložení diagramu	29
6.7	Organizace kódu	30
7	Implementace algoritmů	31
7.1	C++ kód	31
7.2	Benchmark implementací algoritmu In-Close	31

8	Vizualizace rozsáhlých dat	33
8.1	Virtualizace a líné načítání	33
8.2	Možnosti vykreslování Hasseova diagramu	34
8.3	Interaktivní vizualizace Hasseova diagramu	34
8.3.1	Geometrie a materiál	35
8.3.2	Omezení počtu trojúhelníků	35
8.3.3	Instanced rendering	35
8.3.4	Další optimalizace	36
8.4	Exportovaná vizualizace Hasseova diagramu	37
8.5	Limity nástroje	37
9	Možnosti nástroje	39
9.1	Nahrání a zobrazení formálního kontextu	39
9.2	Interaktivní vizualizace konceptuálního svazu	39
9.3	Export vizualizace konceptuálního svazu	40
9.4	Průzkumník sousedství	41
	Závěr	42
	Conclusions	43
	A Zprovozněná aplikace	44
	B Obsah elektronických dat	44
	Literatura	45

Seznam obrázků

1	Konceptuální svaz formálního kontextu z tabulky 1.	10
2	Ukázka nástroje Context Explorer.	11
3	Ukázka nástroje Galicia.	12
4	Ukázka nástroje LatViz.	13
5	Ukázka GUI pro vizualizaci konceptuálního svazu v conexp-clj. . .	13
6	Diagram konceptuálního svazu <i>Live in water</i> [19] s rovnoměrným rozmístěním (vlevo nahoře), rozmístěním do elipsy (vpravo nahoře) a rozmístěním pomocí algoritmu Brandese a Köpfa (dole). .	18
7	Diagram konceptuálního svazu <i>Live in water</i> [19] vygenerovaný pomocí Sugiyama frameworku (vlevo nahoře), Freeseova algoritmu (vpravo nahoře) a algoritmu ReDraw (dole).	20
8	Základní architektura aplikace.	21
9	Komunikace s workerem MainWorker.	24
10	Komunikace s workerem DiagramLayoutWorker.	25
11	WebGL, Three.js, React Three Fiber a Drei.	34
12	Hasseův diagram konceptuálního svazu datasetu <i>Mushroom</i> vykreslený pomocí Canvas API v prohlížeči Google Chrome.	38
13	Hasseův diagram konceptuálního svazu datasetu <i>Mushroom</i> vykreslený pomocí Canvas API v prohlížeči Safari.	38
14	Tabulka formálního kontextu v nástroji konlatt.	39
15	Interaktivní vizualizace konceptuálního svazu v nástroji konlatt. .	40
16	Export Hasseova diagramu konceptuálního svazu v nástroji konlatt.	41
17	Průzkumník sousedství v nástroji konlatt.	41

Seznam tabulek

1	Planety Sluneční soustavy a jejich atributy s jedním vyznačeným formálním konceptem.	9
2	Srovnání implementací algoritmu In-Close v různých prostředích. .	32

Seznam zdrojových kódů

1	Pseudokód minimalizace křížení hran.	17
2	Úložiště useNewProjectStore.	27
3	With-transformace withLayoutBox.	27
4	Typ FormalContext.	28
5	Typ FormalConcept.	28
6	Typ ConceptLattice.	29
7	Typ ConceptLatticeLayout.	30
8	Vlastní React Three Fiber komponenta Links.	36

1 Úvod

Formální konceptuální analýza (FCA) je jednou z metod pro analýzu objekt-atributových dat, která umožňuje popsat vztahy mezi objekty a jejich atributy [1]. Jedním z hlavních výstupů je *konceptuální svaz*, tedy hierarchicky uspořádaná množina pojmů, tzv. *formálních konceptů*, vyskytujících se ve zkoumaných datech. Aby byly informace obsažené v konceptuálním svazu dobře srozumitelné pro člověka, je nezbytná vhodná vizualizace tohoto svazu, nejčastěji formou Hasseova diagramu.

Přestože existuje široká škála nástrojů umožňující aplikaci formální konceptuální analýzy a vizualizaci konceptuálních svazů, jejich praktická využitelnost je omezená. Dostupné nástroje jsou často technicky zastaralé, uživatelsky nepřívětivé nebo generují pouze statické diagramy bez možnosti interakce. Výsledkem je, že metody formální konceptuální analýzy zůstávají dostupné jen pro úzkou skupinu technicky zdatných uživatelů.

Hlavním cílem této diplomové práce je učinit formální konceptuální analýzu přístupnější širší škále uživatelů prostřednictvím nového online nástroje *konlatt*¹. Při tvorbě nástroje jsem kladl důraz především na jeho uživatelskou přívětivost, interaktivitu, výkon a dlouhodobou udržitelnost.

Text práce začíná krátkým shrnutím základních pojmů formální konceptuální analýzy. Dále je uveden přehled existujících nástrojů, na něž navazuje stanovení konkrétních požadavků na nový nástroj. Následující části jsou věnovány popisu zvolených algoritmů pro výpočet rozložení prvků Hasseova diagramu a popisu celkové architektury aplikace včetně klíčových implementačních detailů. Nakonec je uveden stručný popis funkcionality, kterou nástroj nabízí.

¹Nástroj je dostupný na <https://radekvym.github.io/konlatt>

2 Základy formální konceptuální analýzy

Formální konceptuální analýza zkoumá objekt-atributová data [1]. Tato data lze popsat trojicí $\langle X, Y, I \rangle$, kde X a Y jsou neprázdné množiny objektů a atributů a I je binární relace mezi množinami X a Y , tedy $I \subseteq X \times Y$. Trojice $\langle X, Y, I \rangle$ se označuje jako *formální kontext*. Příklad formálního kontextu je uveden v tabulce 1. Řádky tabulky zde představují objekty a sloupce jejich atributy.

	Small	Medium	Large	Close	Far	Moon	No moon
Mercury	×			×			×
Venus	×			×			×
Earth	×			×		×	
Mars	×			×		×	
Jupiter			×		×	×	
Saturn			×		×	×	
Uranus		×			×	×	
Neptune		×			×	×	

Tabulka 1: Planety Sluneční soustavy a jejich atributy s jedním vyznačeným formálním konceptem.

2.1 Formální koncept

Cílem formální konceptuální analýzy je z objekt-atributových dat extrahovat tzv. formální koncepty, případně i jejich hierarchickou strukturu.

Vzhledem k formálnímu kontextu $\langle X, Y, I \rangle$ je možné definovat dva operátory $\uparrow : 2^X \rightarrow 2^Y$ a $\downarrow : 2^Y \rightarrow 2^X$ s předpisem

$$A^\uparrow = \{y \in Y \mid \text{pro každý } x \in A : \langle x, y \rangle \in I\}$$

pro $A \subseteq X$ a

$$B^\downarrow = \{x \in X \mid \text{pro každý } y \in B : \langle x, y \rangle \in I\}$$

pro $B \subseteq Y$.

Formální koncept je dvojice $\langle A, B \rangle$, kde pro $A \subseteq X$ a $B \subseteq Y$ platí, že $A^\uparrow = B$ a $B^\downarrow = A$. Množina A se označuje jako *extent* a množina B jako *intent* konceptu. V tabulce 1 je zvýrazněn formální koncept s extensem $\{Earth, Mars\}$ a intentem $\{Small, Close, Moon\}$.

2.2 Konceptuální svaz

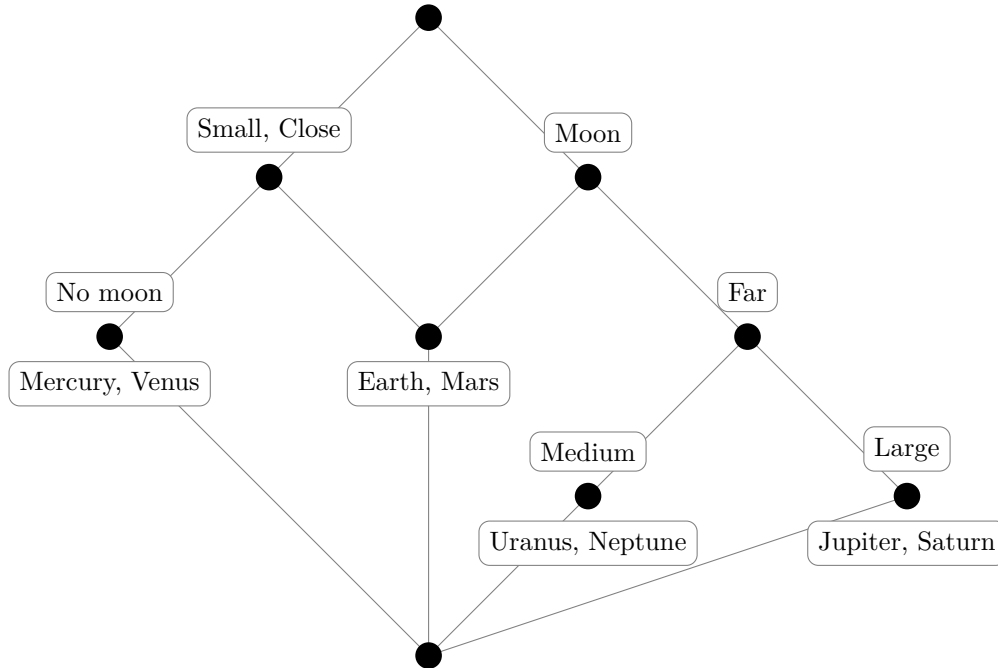
Zmíněná hierarchická struktura formálních konceptů je popsána binární relací uspořádání $\leq$ definovanou na množině všech konceptů $\mathcal{B}(\langle X, Y, I \rangle)$ v kontextu $\langle X, Y, I \rangle$ s předpisem:

$$\langle A_1, B_1 \rangle \leq \langle A_2, B_2 \rangle \text{ právě když } A_1 \subseteq A_2 \text{ (nebo ekvivalentně } B_2 \subseteq B_1)$$

Množina $\mathcal{B}(\langle X, Y, I \rangle)$ spolu s relací uspořádání $\leq$ tvoří úplný svaz a označuje se jako *konceptuální svaz*. Relaci uspořádání $\leq$ lze interpretovat jako vztah podkoncept-nadkoncept. Ve vztahu $\langle A_1, B_1 \rangle \leq \langle A_2, B_2 \rangle$ je koncept $\langle A_1, B_1 \rangle$ podkonceptem (je konkrétnější) konceptu $\langle A_2, B_2 \rangle$, respektive koncept $\langle A_2, B_2 \rangle$ je nadkonceptem (je obecnější) konceptu $\langle A_1, B_1 \rangle$.

Konceptuální svazy lze vizualizovat pomocí Hasseových diagramů. Hasseův diagram konceptuálního svazu formálního kontextu z tabulky 1 je možné vidět na obrázku 1. Každý vrchol reprezentuje formální koncept. Pro přehlednost se v diagramech využívá tzv. *redukované značení* – popisky pod vrcholy jsou objektové popisky a popisky nad vrcholy jsou atributové popisky. Extent a intent konkrétního konceptu vyčteme z diagramu dle Hlavní věty o konceptuálních svazech následovně:

- Extent konceptu získáme přečtením všech objektových popisků z dolního kužele daného konceptu.
- Intent konceptu získáme přečtením všech atributových popisků z horního kužele daného konceptu.



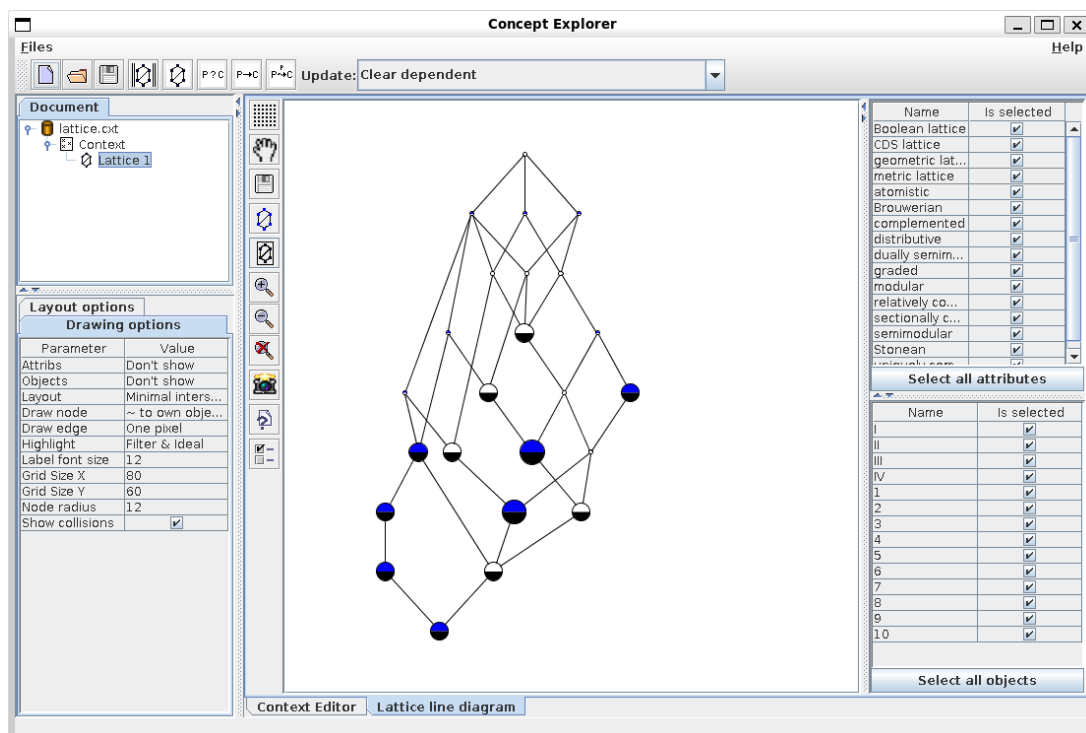
Obrázek 1: Konceptuální svaz formálního kontextu z tabulky 1.

3 Existující řešení

Od vzniku formální konceptuální analýzy byla vyvinuta široká škála nástrojů implementujících její metody. Z uživatelského hlediska lze tyto implementace rozdělit do dvou hlavních kategorií: aplikace s grafickým uživatelským rozhraním (GUI) a programové knihovny. Obě kategorie nabízejí nástroje umožňující konstrukci nebo import konceptuálních svazů, jejich vizualizaci formou Hasseových diagramů a následnou analýzu.

3.1 GUI aplikace

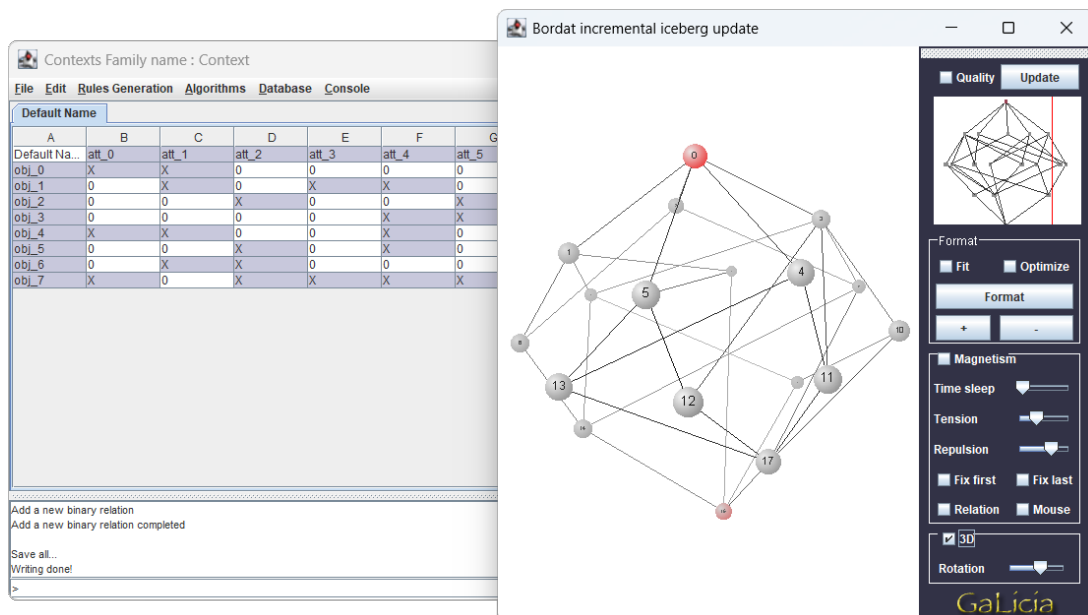
Jedním z nejznámějších zástupců aplikací s grafickým rozhraním je desktopový nástroj **Concept Explorer (ConExp)**, poprvé představený v roce 2000 [2]. Jeho ukázka je uvedena na obrázku 2. Jelikož je tento nástroj postaven na starších verzích platformy Java a již není udržován, může být jeho zprovoznění pro nezkušené uživatele problematické [3]. Proto později vznikly novější implementace tohoto nástroje – především **ConExp NG** a **ConExp FX**.



Obrázek 2: Ukázka nástroje Context Explorer.

Mezi další významné, avšak dnes rovněž neudržované projekty, patří sada nástrojů **ToscanaJ** [4] a nástroj **Galicja** [5] (ukázka na obrázku 3). Přínosem

nástroje Galicia je možnost výběru z různých algoritmů pro kreslení konceptuálních svazů (tj. výpočet rozložení vrcholů Hasseova diagramu) a zároveň možnost konfigurace parametrů těchto algoritmů. Jedinečnou funkcionalitou je taktéž možnost 3D zobrazení diagramu. Opět jde o desktopové nástroje a jejich instalace a spuštění naráží na podobné problémy jako Concept Explorer.



Obrázek 3: Ukázka nástroje Galicia.

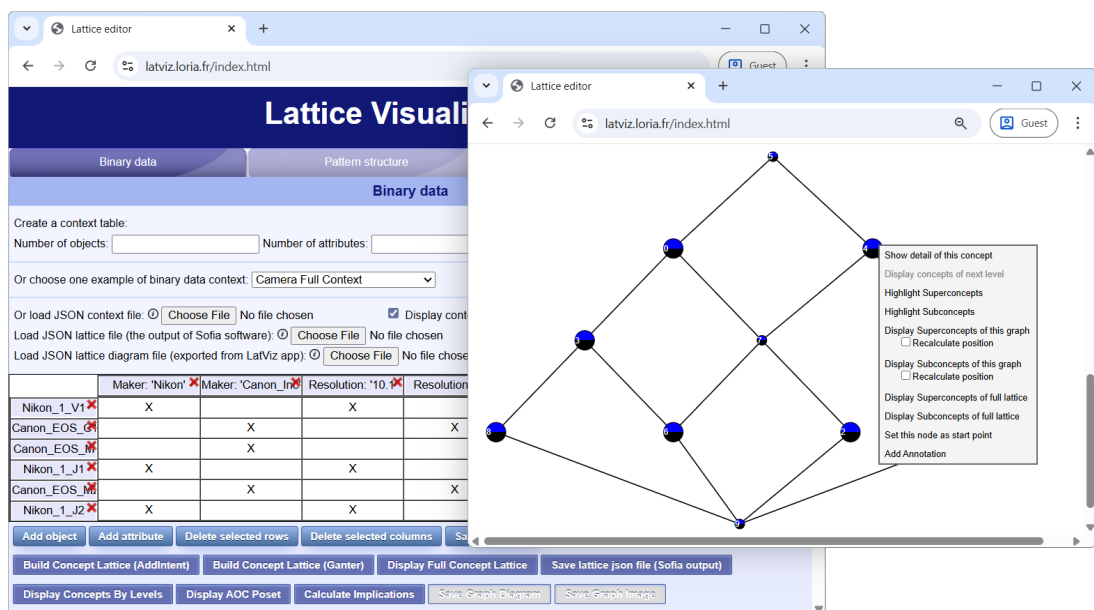
Ve webovém prostředí lze použít nástroj **LatViz** [6] (ukázka na obrázku 4). Podobně jako Galicia i LatViz nabízí výběr z více algoritmů pro kreslení konceptuálních svazů. Navíc však přichází s dalšími metodami jejich vizualizace, například zobrazení po jednotlivých úrovních Hasseova diagramu nebo vizualizace atribut-objektové částečně uspořádané množiny konceptů (AOC poset).

Dále stojí za zmínku ještě webový nástroj **FCA Tools Bundle** [7], který slouží zároveň i jako platforma pro sdílení formálních kontextů. Nevýhodou je však nutnost registrace.

3.2 Knihovny

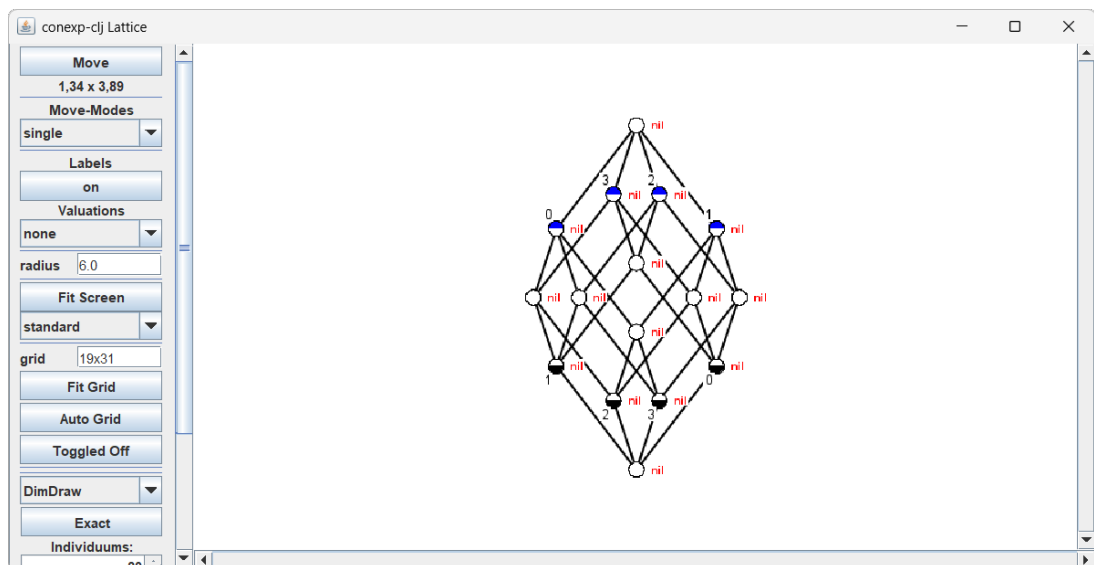
Druhou významnou kategorií tvoří knihovny, které umožňují integraci metod formální konceptuální analýzy do vlastních softwarových řešení nebo jejich použití v interaktivních prostředích (REPL). Tyto nástroje jsou určeny především pro pokročilé uživatele a vyžadují znalost některého programovacího jazyka.

V ekosystému jazyka Python jsou k dispozici knihovny **Galactic** [8] a **Concepts** [9], které poskytují základní datové struktury a algoritmy pro práci s konceptuálními svazy. K jejich vizualizaci využívají software Graphviz.



Obrázek 4: Ukázka nástroje LatViz.

V programovacím jazyce Clojure je dostupná mnohoúčelová knihovna a interaktivní nástroj **conexp-clj** [10]. Knihovna nabízí výběr z několika algoritmů pro kreslení konceptuálních svazů s možností jejich parametrizace. Pro tyto účely dokonce poskytuje i jednoduché GUI (ukázka na obrázku 5).



Obrázek 5: Ukázka GUI pro vizualizaci konceptuálního svazu v conexp-clj.

Pro programovací jazyk R pak ještě existuje knihovna **fcaR** [11] s primárním zaměřením na fuzzy FCA. Vizualizaci konceptuálního svazu umožňuje přímo ve vývojovém prostředí.

3.3 Nedostatky uvedených řešení

Z uvedeného výčtu, který jistě není úplný, vyplývá, že nástroje umožňující vizualizaci konceptuálních svazů existují. Jejich praktické využití však naráží na několik zásadních nedostatků. Mezi nejvýznamnější nedostatky patří:

- Technická zastaralost – Desktopové nástroje jsou postaveny na zastaralých technologiích, což výrazně ztěžuje jejich zprovoznění a také stabilitu.
- Nutnost znalosti programovacího jazyka – Uvedené knihovny sice nabízejí vysokou flexibilitu, vyžadují však znalost programovacího jazyka a specifického API dané knihovny.
- Nízká uživatelská přívětivost – Grafická uživatelská rozhraní často nerespektují moderní standardy a mají zastaralý vizuální styl. Z pohledu dnešního uživatele tak mohou působit nepřehledně a být náročná na pochopení.
- Absence interaktivity – Některé z uvedených nástrojů generují pouze statické diagramy a neumožňují interaktivní analýzu konceptuálního svazu.

Kvůli těmto nedostatkům jsou stávající řešení použitelná pouze pro úzkou skupinu technicky zdatných uživatelů. To je zásadní překážka pro širší využití metod formální konceptuální analýzy v jiných než informatických oborech.

4 Požadavky na nástroj

Mým cílem bylo vytvořit nástroj pro vizualizaci konceptuálních svazů, který nebude trpět nedostatky existujících řešení, a učiní tak formální konceptuální analýzu přístupnější širší škále uživatelů. Mezi základní, přirozeně očekávanou funkcionalitu takového nástroje patří:

- import formálního kontextu ve standardních formátech – Burmeister, CSV, JSON a XML,
- zobrazení formálního kontextu formou tabulky,
- zobrazení přehledu všech formálních konceptů daného kontextu,
- vizualizace konceptuálního svazu formou Hasseova diagramu,
- export daného Hasseova diagramu v různých rastrových a vektorových formátech – PNG, JPEG, SVG a TikZ.

Výsledný nástroj záměrně neumožňuje vytváření nebo editaci formálních kontextů. Namísto vývoje této funkcionality jsem se rozhodl věnovat své úsilí samotné vizualizaci konceptuálního svazu formou Hasseova diagramu, která je pro tento typ nástroje klíčová. Editační funkce by se mohly implementovat v rámci případného budoucího rozvoje nástroje. Pro vizualizační část nástroje jsem si stanovil následující požadavky:

- automatické generování rozložení vrcholů diagramu s možností výběru z několika různých algoritmů a možností jejich parametrizace,
- možnost manipulace s jednotlivými vrcholy diagramu pro případné ruční úpravy rozložení,
- možnost diagram libovolně přibližovat, posouvat a otáčet,
- možnost zobrazit diagram ve 2D nebo 3D režimu,
- zobrazení redukovaného objektového a atributového značení s možností skrýt některé popisky,
- zobrazení detailu konceptu při najetí nebo výběru odpovídajícího vrcholu,
- zvýraznění nebo zobrazení zvoleného podsvazu,
- zobrazení pohledu zaměřeného na okolí konkrétního konceptu,
- zvýraznění vrcholů a hran na základě filtrů nebo vyhledávání.

Důraz jsem kladl na uživatelskou přívětivost nástroje. Požadoval jsem, aby respektoval moderní standardy uživatelských rozhraní, byl přehledný, intuitivní a dostatečně výkonný. Z implementačního hlediska bylo mým cílem vytvořit dobře udržitelný kód s minimem závislostí na externích knihovnách, to vše při zachování příjemného vývojářského zážitku.

5 Použité algoritmy

V rámci nástroje jsem implementoval dva algoritmy z oblasti formální konceptuální analýzy. Pro výpočet všech konceptů z formálního kontextu je použita první verze algoritmu **In-Close** [12]. Navazující výpočet relace pokrytí konceptuálního svazu zajišťuje algoritmus **Concepts Cover** [13]. Tyto algoritmy jsem zvolil pro jejich příznivou časovou složitost a relativní jednoduchost implementace.

Pro automatické kreslení Hasseových diagramů, tedy výpočet rozložení jejich vrcholů, jsem se rozhodl implementovat následující metody:

- **Sugiyama framework** – rychlá a osvědčená metoda produkující přehledné rozložení vrcholů acyklických orientovaných grafů,
- **Freeseův algoritmus** – force-directed metoda pro kreslení svazů,
- **ReDraw** – novější force-directed metoda pro kreslení svazů.

Následující podkapitoly se zaměřují především na specifika implementace těchto metod v mém nástroji. Detailní popis metod lze nalézt v citované literatuře. V rámci budoucího rozvoje nástroje se nabízí rozšíření o další metody kreslení nebo rozšíření těch stávajících.

5.1 Sugiyama framework

Sugiyama framework je jednou z nejrozšířenějších metod pro kreslení acyklických orientovaných grafů [14, 15]. Cílem této metody je rozdělit vrcholy grafu do vrstev tak, aby hrany směřovaly pouze jedním směrem. Tato metoda se proto označuje také jako vrstevná (layered) nebo hierarchická. Algoritmus standardně probíhá ve čtyřech fázích:

1. Rozdělení vrcholů do vrstev – Vrcholy jsou rozděleny do horizontálních vrstev, čímž jsou určeny jejich vertikální souřadnice.
2. Přidání falešných vrcholů – Hrany, které procházejí přes více vrstev, jsou nahrazeny cestami falešných vrcholů a hran tak, aby každá hrana v rozšířeném grafu spojovala dva vrcholy ze sousedních vrstev.
3. Minimalizace křížení hran – Vrcholy každé vrstvy jsou seřazeny tak, aby se snížil počet křížení hran, které danou vrstvu spojují s tou předchozí.
4. Určení horizontálních souřadnic vrcholů – Podle uspořádání vrcholů z předchozí fáze jsou určeny jejich finální horizontální souřadnice.

5.1.1 Rozdělení vrcholů do vrstev

Pro rozdělení vrcholů do vrstev jsem zvolil metodu nejdelší cesty. Nejprve je umístěn největší vrchol svazu do první vrstvy. Každý zbývajících vrchol je pak umístěn do d -té vrstvy, kde d je délka nejdelší cesty z daného vrcholu do největšího vrcholu.

5.1.2 Minimalizace křížení hran

Problém nalezení uspořádání vrcholů s minimálním počtem křížení hran je NP-těžký, proto se v praxi využívají různé heuristické metody. Rozhodl jsem se použít metodu barycentra (barycenter method), která určuje pozici vrcholu na základě aritmetického průměru pozic jeho sousedů v sousední vrstvě. Metodu lze snadno implementovat, má nízkou časovou složitost a dosahuje kvalitních výsledků [16].

Tuto metodu aplikuji na vrstvy diagramu opakovaně, dokud nenastane jedna z následujících možností:

- je dosaženo nulového počtu křížení hran,
- počet křížení hran dosažený v aktuální iteraci je stejný nebo větší než v předchozí iteraci,
- je dosaženo maximálního povoleného počtu iterací.

Tento postup je detailněji zachycen v pseudokódu 1. Výpočet počtu křížení hran zajišťuje algoritmus založený na počítání inverzí v permutaci [17].

```
1 reduceCrossings(layers):
2     bestLayers ← applyBarycenter(layers)
3     bestCount ← crossCount(bestLayers)
4     lastCount ← bestCount
5
6     for i ← 1 to MAX_ITERATIONS_COUNT do
7         if bestCount = 0
8             break
9         lastLayers ← applyBarycenter(bestLayers)
10        newCount ← crossCount(lastLayers)
11        if newCount >= lastCount
12            break
13        if newCount < bestCount
14            bestCount ← newCount
15            bestLayers ← lastLayers
16        lastCount ← newCount
17
18    return bestLayers
```

Zdrojový kód 1: Pseudokód minimalizace křížení hran.

Pro zvýšení stability výsledku se každá iterace metody barycentra (volání `applyBarycenter` v pseudokódu 1) skládá ze tří kroků:

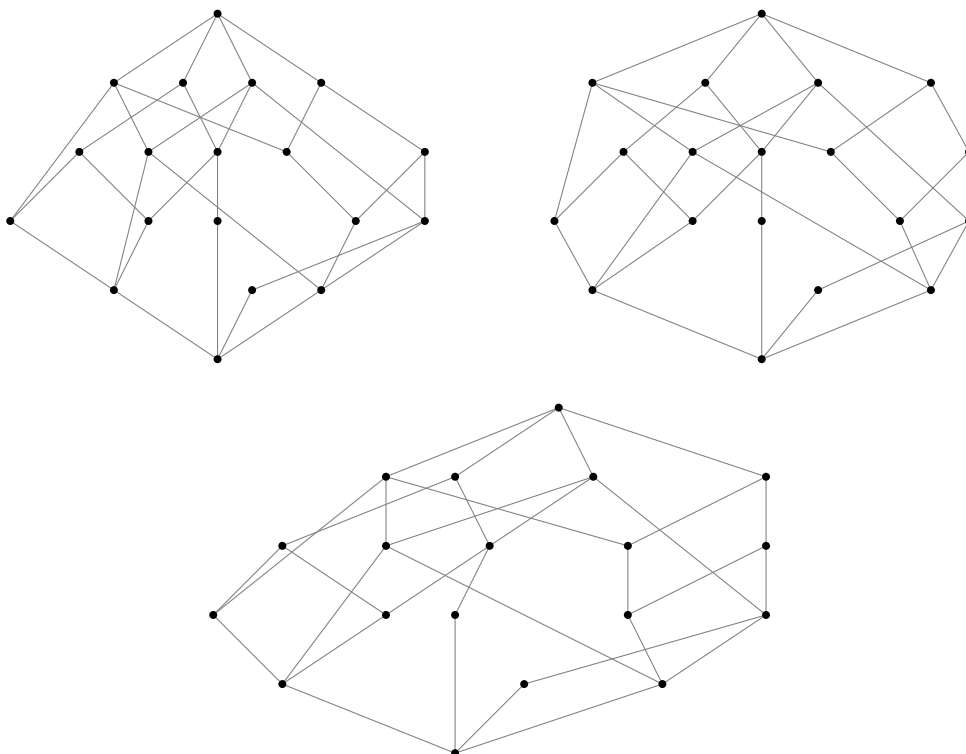
1. průchod shora dolů – výpočet pozic podle horních sousedů,
2. průchod zdola nahoru – výpočet pozic podle dolních sousedů,
3. kombinovaný průchod – výpočet pozic podle horních i dolních sousedů.

5.1.3 Určení horizontálních souřadnic vrcholů

V nástroji jsem implementoval celkem tři různé metody pro výpočet horizontálních souřadnic vrcholů diagramu, které uživateli umožňují zvolit vhodný vzhled:

- **Rovnoměrné rozmístění** – Metoda, která rozmísťuje vrcholy tak, aby mezi nimi byly stejné rozestupy. Jednotlivé vrstvy jsou následně horizontálně zarovnané na střed, což vytváří symetrický vzhled.
- **Rozmístění do elipsy** – Mnou navržená metoda, která rozmísťuje vrcholy rovnoměrně tak, aby výsledný tvar diagramu přibližně kopíroval hranice elipsy. Tento přístup efektivně využívá prostor a zabráňuje nadměrnému shlukování vrcholů.
- **Algoritmus Brandese a Köpfa** – Metoda, která se zaměřuje na zarovnání vrcholů s jejich sousedy a minimalizaci ohybů hran [18]. Výsledkem je rozložení s dlouhými vertikálními úseky.

Vzhledem k tomu, že jsou lomené hrany ve vizualizacích svazů obvykle nežádoucí, jsou po určení horizontálních souřadnic falešné vrcholy odebrány a nahrazeny původními hranami. Falešné vrcholy tedy pouze napomáhají k dosažení co nejlepšího rozložení. Ukázka výstupů těchto metod je uvedena na obrázku 6.



Obrázek 6: Diagram konceptuálního svazu *Live in water* [19] s rovnoměrným rozmístěním (vlevo nahoře), rozmístěním do elipsy (vpravo nahoře) a rozmístěním pomocí algoritmu Brandese a Köpfa (dole).

5.2 Freeseův algoritmus

Freeseův algoritmus patří do skupiny tzv. force-directed algoritmů, tedy algoritmů založených na silovém modelu [20]. Graf je zde chápán jako systém, mezi jehož vrcholy (prvky) působí přitažlivé a odpudivé síly.

Algoritmus nejprve určí tzv. *rank* funkci, která definuje výšku (souřadnici y) jednotlivých vrcholů ve trojrozměrném prostoru. Vrcholy ve stejné výšce jsou umístěné kolem kružnice v odpovídající xz rovině. Na vrcholy jsou následně opakovaně aplikovány síly pro určení jejich finálních souřadnic x a z . Přitažlivé síly působí mezi porovnatelnými prvky svazu a odpudivé síly mezi neporovnatelnými prvky svazu ve stejné výšce. Síly se takto na vrcholy aplikují ve třech fázích s různou intenzitou. V první fázi jsou intenzivnější odpudivé síly, ve druhé přitažlivé síly a v poslední fázi jsou síly vyrovnané. Nakonec se provede projekce z trojrozměrného prostoru do dvojrozměrného.

Ve své implementaci jsem použil stejné parametry algoritmu (například intenzita sil), jaké použil autor v jeho původní implementaci [21]. Projekci do dvojrozměrného prostoru realizuji vynecháním souřadnice z (hloubky).

5.3 ReDraw

Algoritmus ReDraw je dalším zástupcem force-directed algoritmů. Na rozdíl od Freeseova algoritmu, který používá silový model pouze pro určení souřadnic x a z vrcholů, ReDraw používá silový model pro určení všech souřadnic vrcholů [22].

Hlavním rysem algoritmu je, že výpočet začíná v prostoru s vysokou dimenzí (větší nebo rovnou dvěma). Zpočátku je vertikální souřadnice y určena na základě topologického uspořádání vrcholů a zbylé souřadnice jsou vygenerovány náhodně. Následně algoritmus opakovaně provádí následující tři kroky, než dosáhne požadované dimenze:

1. Node step – Pomocí force-directed přístupu optimalizuje rozmístění vrcholů na základě jejich vzájemné polohy.
2. Line step – Pomocí force-directed přístupu optimalizuje vzdálenosti, vzájemné úhly a paralelismus mezi hranami.
3. Redukce dimenze – Provádí projekci do prostoru s nižší dimenzí pomocí analýzy hlavních komponent (PCA), přičemž striktně zachovává vertikální souřadnici y . Po dosažení požadované dimenze je tento krok vynechán.

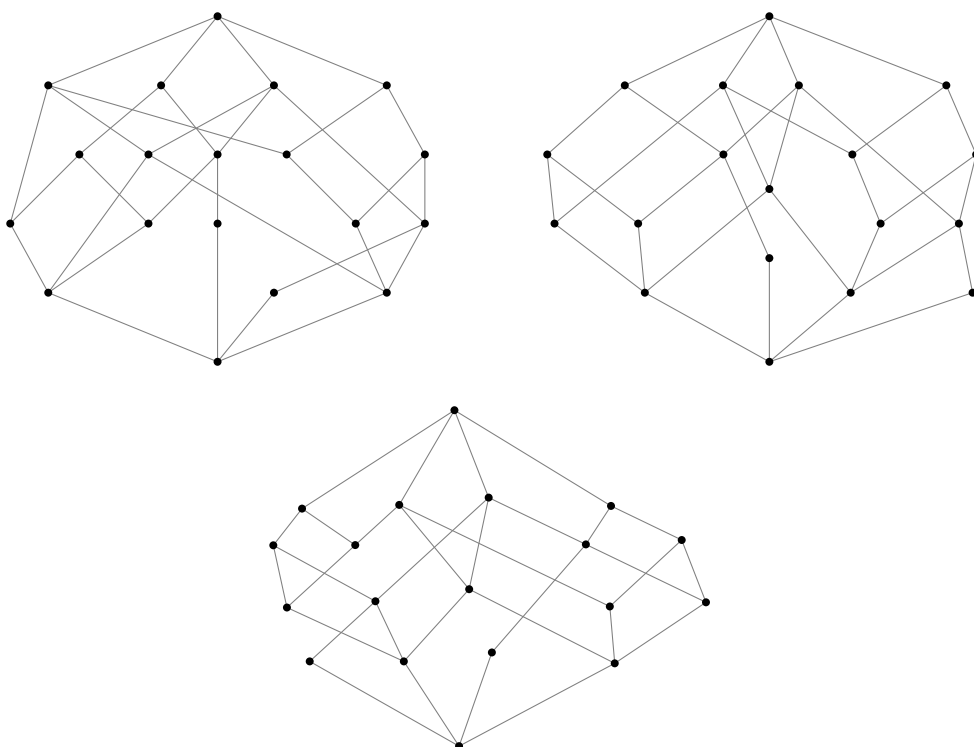
Všechny kroky jsou navrženy tak, aby byl vždy dodržen tzv. *vertikální invariant*, který zaručuje, že větší prvek má vyšší souřadnici y než prvek menší. Má implementace algoritmu je silně inspirovaná původní implementací jeho autorů [23]. Výpočet začíná v páté dimenzi a postupně probíhá redukce do druhé nebo třetí dimenze dle volby uživatele. Uživatel má v nástroji zároveň možnost si zvolit, zda má být druhý krok algoritmu (line step) vynechán.

5.4 Srovnání metod pro kreslení Hasseových diagramů

Všechny tři použité metody generují pro menší diagramy uspokojivá rozložení. Hodnocení výstupů těchto metod na základě uživatelské studie provedli například Dominik Dürschnabel a Gerd Stumme [22]. Ačkoliv se jejich implementace metod mohou v dílčích krocích (například volbou heuristiky) lišit od těch mých, i tak považuji jejich výsledky v tomto případě za relevantní. Srovnání výstupů mých implementací pro jeden konkrétní dataset je uvedeno na obrázku 7.

Metody se liší nejen svým přístupem k danému problému, ale i svou časovou složitostí. Díky zvolené heuristice barycentra je nejefektivnější metodou Sugiyama framework, který zvládá i velmi rozsáhlé diagramy (až statisíce vrcholů). Takové diagramy ale bývají z důvodu vysoké šířky vrstev a hustoty hran prakticky nečitelné. Právě pro svou nízkou výpočetní náročnost je v mém nástroji Sugiyama framework nastaven jako výchozí metoda.

Force-directed algoritmy Freese a ReDraw jsou kvůli povaze silového modelu výpočetně náročnější. Tyto metody jsou proto použitelné pouze pro menší diagramy. Výrazně pomalejší je zejména algoritmus ReDraw, a to hlavně kvůli kroku line step, jehož časová složitost je $O(n^4)$, kde n je počet vrcholů [22]. Jednou ze specifických vlastností těchto algoritmů je možnost výpočtu rozložení ve trojrozměrném prostoru, čehož můj nástroj využívá a takové zobrazení uživateli umožňuje.



Obrázek 7: Diagram konceptuálního svazu *Live in water* [19] vygenerovaný pomocí Sugiyama frameworku (vlevo nahoře), Freeseova algoritmu (vpravo nahoře) a algoritmu ReDraw (dole).

6 Architektura aplikace

Dříve zmíněné požadavky na nástroj byly rozhodujícím faktorem při volbě použitých technologií i architektury aplikace. Nástroj jsem se rozhodl implementovat jako statickou jednostránkovou webovou aplikaci (SPA). Stěžejní výhodou tohoto řešení je, že kód webových aplikací díky stabilitě webových standardů a vysoké zpětné kompatibilitě prohlížečů zůstává dlouhodobě funkční, na rozdíl od mnohých desktopových aplikací, které často narážejí na nekompatibilitu s vyvíjejícími se operačními systémy nebo běhovými prostředími.

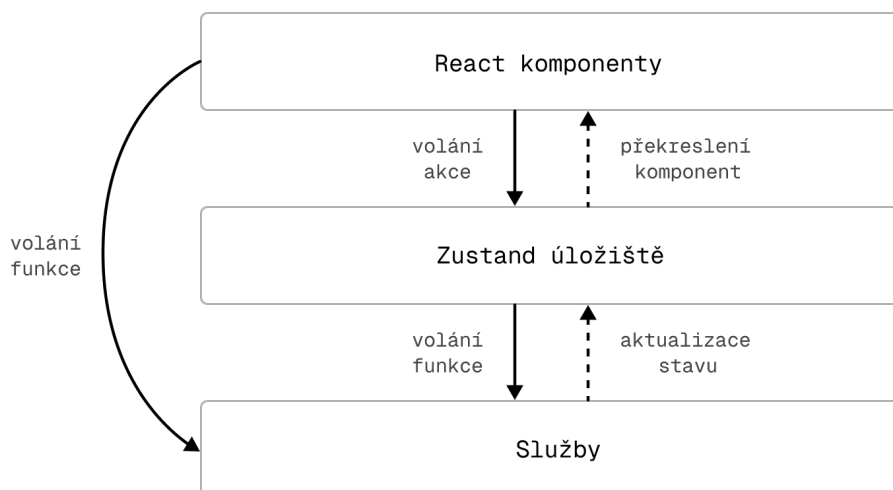
Veškerá aplikační logika a vykreslování uživatelského rozhraní probíhá ve webovém prohlížeči na straně klienta. Server funguje pouze jako úložiště statických souborů (zejména HTML, JavaScript, CSS). To přináší dva benefity:

- Škálovatelnost a dostupnost – Nástroj lze provozovat na platformách jako GitHub Pages zcela bez nákladů a nároků na údržbu.
- Bezpečnost – Jelikož data zpracovává přímo prohlížeč, potenciálně citlivé informace neopouštějí zařízení uživatele.

Ke tvorbě uživatelského rozhraní jsem zvolil populární knihovnu React [24], čemuž je uzpůsobena celková architektura aplikace. Základními stavebními bloky architektury jsou:

- React komponenty – Tvoří uživatelské rozhraní.
- Zustand úložiště – Slouží jako prostředníci spravující globální stav aplikace.
- Služby – Zapouzdřují aplikační logiku.

Způsob interakce mezi těmito bloky je znázorněn na obrázku 8.



Obrázek 8: Základní architektura aplikace.

6.1 Základní technologie

Většina zdrojového kódu je napsána v jazyce TypeScript. Jeho statická kontrola typů umožnila odhalit množství chyb již při psaní kódu a minimalizovat tak problémy za běhu aplikace. Výpočetně náročné algoritmy jsou implementovány v jazyce C++ a kompilovány do WebAssembly (podrobněji v kapitole 7). Při dokumentování kódu jsem využil nástroj Google Gemini.

Pro sestavování aplikace jsem zvolil moderní nástroj Vite [25], který je v současnosti de facto standardem pro vývoj jednostránkových React aplikací. Součástí ekosystému Vite je také testovací framework Vitest [26]. Ten jsem využil pro automatizované testování kódu.

Technologie Vite taktéž výrazně usnadnila implementaci podpory funkcionality Progressive Web Apps (PWA) [27], a to pomocí balíčku Vite Plugin PWA [28]. Z prohlížečů, které podporují tuto funkcionalitu, je tedy možné si nástroj nainstalovat a používat jej podobně jako nativní desktopovou aplikaci.

6.2 Uživatelské rozhraní

Základem celé aplikace jsou funkcionální React komponenty, které tvoří její uživatelské rozhraní. Koncept React komponent mi umožnil rozdělit uživatelské rozhraní na menší, samostatné a znovupoužitelné části. Funkcionální komponenty využívají pro sdílení logiky tzv. *React hooks* – funkce, které se volají přímo v těle komponenty a zapouzdřují konkrétní chování. Lokální stav je v komponentách spravován pomocí hooku `useState()`. Pro správu komplexnějšího globálního stavu jsem zvolil knihovnu Zustand (více v kapitole 6.5).

Jelikož se jedná o jednostránkovou aplikaci, je navigace mezi jednotlivými stránkami řešena na straně klienta pomocí knihovny React Router [29]. Ta definuje hierarchii URL cest (*routes*), které odpovídají jednotlivým stránkám aplikace. Aplikace je tvořena celkem čtyřmi stránkami:

- Úvodní stránka – Slouží jako vstupní bod aplikace, kterému dominuje vstupní pole pro nahrání analyzovaného datasetu.
- Průzkumník kontextu – Poskytuje zobrazení formálního kontextu v podobě interaktivní tabulky s možností filtrace objektů a atributů.
- Vizualizace svazu – Obsahuje interaktivní vizualizaci Hasseova diagramu konceptuálního svazu a přehled všech formálních konceptů.
- Průzkumník sousedství – Spolu s přehledem všech formálních konceptů nabízí pohled zaměřený na okolí konkrétního konceptu.

Mimo tyto stránky aplikace ještě obsahuje několik modálních dialogů. Jde především o dialogy určené pro výběr datasetu a export dat. Všechny dialogy jsou postaveny na HTML elementu `HTMLDialogElement`, čímž odpadá nutnost použití externích knihoven. Hlášení chyb uživateli jsem vyřešil pomocí vlastního systému toast notifikací. Ten využívá moderní Popover API, které umožňuje

zobrazovat elementy nad veškerým obsahem aplikace. Pro zobrazení jednoduché toast notifikace v aplikaci stačí zavolat funkci `toast()` s požadovaným textem.

6.2.1 Vizualní podoba uživatelského rozhraní

Vizuální podoba uživatelského rozhraní je postavena primárně na utility-first CSS frameworku Tailwind CSS [30]. Díky němu jsem napsal minimum CSS kódu a nemusel se soustředit na jeho organizaci a ani vhodné pojmenovávání tříd. V souvislosti s integrací Tailwind CSS do React aplikace dále využívám knihovny `tailwind-merge` [31] a `clsx` [32] pro bezpečnou kombinaci a dynamické přepínání CSS tříd na základě stavu komponenty.

Všechny ikony v aplikaci poskytl knihovna React Icons [33], která sdružuje širokou škálu ikon z množství sad. Pro udržení konzistentního vizuálního jazyka jsem volil všechny ikony pouze z jedné sady – Lucide.

6.3 Služby

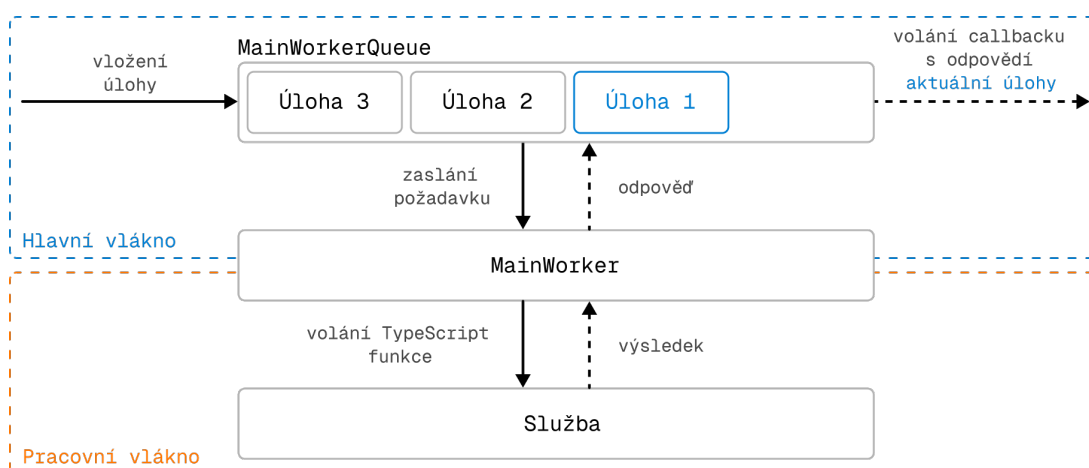
V rámci architektury aplikace jsou jako *služby* označovány sady funkcí, které poskytují nějakou ucelenou funkcionalitu, typicky zpracování dat. V aplikaci jsou čtyři základní skupiny služeb poskytující následující funkcionality:

- Parsování vstupních dat – Vstupem těchto služeb je vždy obsah souboru (datasetu) nahraného uživatelem a výstupem interní reprezentace datových struktur (formální kontexty, koncepty a konceptuální svazy), jejichž popis se v daném souboru nachází. Pro práci s XML formáty jsem zvolil knihovnu `fast-xml-parser` [34].
- Výpočet základních datových struktur – Jedná se o služby zapouzdřující implementace algoritmů pro výpočet konceptuálního svazu a přidružených datových struktur. Konkrétně jde o výpočet všech formálních konceptů z předaného kontextu, relace pokrytí svazu a redukováného objektového a atributového značení (labeling).
- Kreslení konceptuálního svazu – Služby zapouzdřující implementace algoritmů pro výpočet rozložení vrcholů Hasseova diagramu.
- Export – Služby zajišťující převod interní reprezentace datových struktur do exportních textových formátů (například JSON, XML, CSV a další).

6.4 Paralelní zpracování

Výpočet základních datových struktur a rozložení Hasseova diagramu jsou operace s vysokou výpočetní náročností. Aby nedocházelo k blokování hlavního vlákna a byla zachována plynulost uživatelského rozhraní, jsou tyto operace vykonávány v separátním pracovním vlákne pomocí Web Workers API. Web worker spravující toto vlákno označuji jako `MainWorker`.

Samotné řízení komunikace s workerem `MainWorker` zajišťuje instance třídy `MainWorkerQueue`. Jedná se o implementaci fronty úloh, která koordinuje asynchronní zasílání požadavků na worker a následný příjem odpovědí tak, aby byly jednotlivé úlohy zpracovávány sekvenčně. Pokud jsou do fronty vloženy například požadavky na výpočet formálních konceptů a bezprostředně poté na výpočet relace pokrytí, je díky této frontě zaručeno, že výpočet relace započne až po úspěšném dokončení výpočtu konceptů. Celý proces je znázorněn na obrázku 9. Komunikaci se samotnou instancí `MainWorkerQueue` řeší výhradně služby označované jako *triggers*, tedy funkce s prefixem `trigger`.



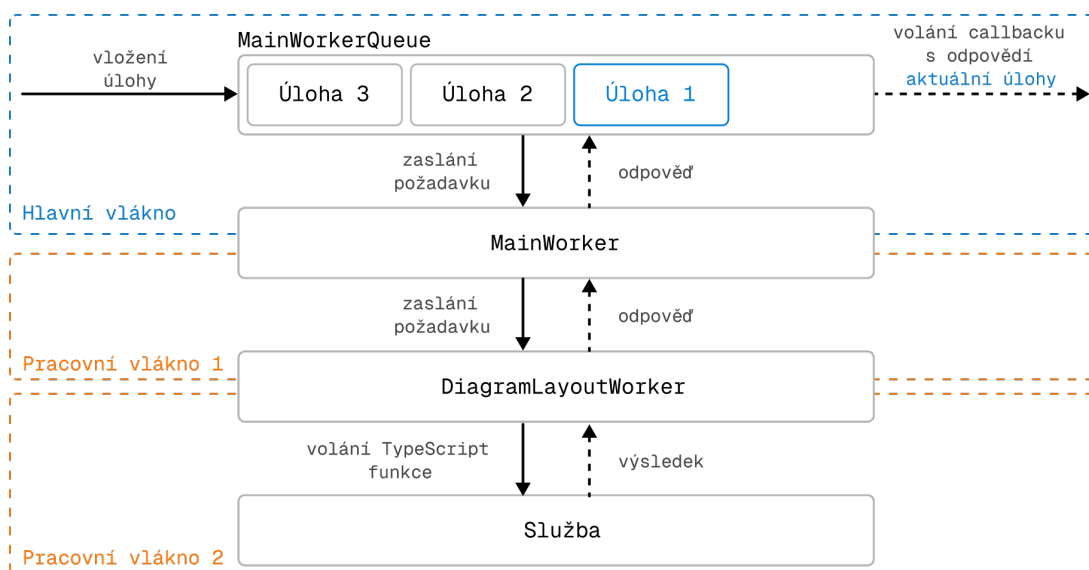
Obrázek 9: Komunikace s workerem `MainWorker`.

6.4.1 Přerušení výpočtu

Nástroj umožňuje uživateli volit parametry rozložení zobrazeného diagramu, například použitý algoritmus. Pokud se uživatel rozhodne tyto parametry změnit, zatímco již probíhá výpočet rozložení s předchozími hodnotami parametrů, je nutné tento výpočet okamžitě ukončit. Proto je každý výpočet rozložení diagramu prováděn v samostatném pracovním vlákne. Web worker spravující toto vlákno označuji jako `DiagramLayoutWorker`.

V případě potřeby ukončení výpočtu je na instanci tohoto workeru zavolána metoda `terminate()`. Tím dojde k okamžitému přerušení vykonávání kódu v pracovním vlákne, aniž by bylo nutné čekat na dokončení aktuální iterace algoritmu. Tento přístup zajišťuje vysokou responzivitu aplikace, neboť nově iniciovaný výpočet může začít okamžitě v čistém prostředí.

O správu instancí workeru `DiagramLayoutWorker` se stará skript workeru `MainWorker`. Celý proces jejich komunikace je znázorněn na obrázku 10.



Obrázek 10: Komunikace s workerem DiagramLayoutWorker.

6.4.2 Optimalizace přenosu dat

Data se mezi vlákny přenášejí kopírováním, což vyžaduje jejich serializaci v jednom vlákne a následnou deserializaci ve druhém vlákne. Tento proces s sebou nese znatelnou režii, zejména při práci s velkými datasety.

Abych tuto režii minimalizoval, udržuji pro aktuálně zpracovávaný dataset vždy právě jeden **MainWorker**, přičemž jsou uchovávána identická data v hlavním i pracovním vlákne současně. Kopie v pracovním vlákne slouží výhradně pro výpočty, zatímco kopie v hlavním vlákne se využívá pro potřeby zobrazení v uživatelském rozhraní. Například po vypočtení všech konceptů se výsledky odesílají do hlavního vlákna pro účely jejich zobrazení. Při následném požadavku na výpočet relace pokrytí se však již neposílají zpět a pracovní vlákno využívá svou vlastní kopii.

Data přenášená mezi vlákny workeru **DiagramLayoutWorker** a workeru **MainWorker** lze jednoduše převést na sekvenci čísel, a reprezentovat je tak jako obecná binární data. V tomto případě proto přenos dat řeším pomocí objektu **ArrayBuffer**, který patří mezi tzv. *přenositelné objekty* (transferable objects). Během přenosu instance **ArrayBuffer** mezi vlákny nedochází ke kopírování samotného obsahu, ale pouze k předání ukazatele na daný buffer. Odesílající vlákno okamžitě ztrácí přístup k těmto datům. Technicky vzato i v tomto případě dochází k určité formě serializace, zásadní výhodou však je, že takto reprezentovaná data lze následně přímo předat do paměti **WebAssembly** modulu bez nutnosti další transformace.

Další možností, jak minimalizovat zmíněnou režii, by potenciálně mohlo být využití objektu **SharedArrayBuffer**. Ten představuje sdílenou paměť, ke

které má hlavní i pracovní vlákno přímý přístup bez nutnosti data kopírovat či serializovat. Jeho použití by však vyžadovalo:

- využít jinou, méně přehlednou datovou reprezentaci datových struktur,
- správně ošetřit atomický přístup ke sdílené paměti,
- zajistit, aby byla stránka v prohlížeči *cross-origin* izolovaná pomocí specifických HTTP hlaviček.

Z důvodu složitější implementace jsem tuto možnost nerealizoval. V budoucnu by však tato cesta mohla být předmětem dalšího zkoumání pro dosažení vyšší efektivity.

6.5 Správa stavu

Aplikace pracuje se značným množstvím globálního stavu. Ten zahrnuje uživatelská nastavení, datové struktury popsané v kapitole 6.6 a také stav některých React komponent, který je nutné zachovat i při navigaci mezi jednotlivými stránkami. K těmto datům navíc vyžaduje přístup řada komponent napříč celou aplikací. Abych správu tohoto stavu udržel centralizovanou a přehlednou, zvolil jsem řešení založené na knihovně Zustand [35].

Zustand funguje na principu centralizovaných úložišť (stores). Jedná se o JavaScriptové objekty skládající se ze dvou hlavních částí:

- vlastnosti (stav) – samotná data,
- akce – funkce, které slouží k modifikaci stavu.

Úložiště stojí zcela mimo strom React komponent, což představuje zásadní výhodu oproti jiným řešením založeným na React Context API. Každá React komponenta se díky tomu může přihlásit k odběru změn pouze specifických částí úložiště. Pokud se změní jiná část úložiště, nedojde k překreslení dané komponenty, což má pozitivní vliv na výkon.

K vytvoření úložiště slouží funkce `create()`, která jako argument přijímá inicializační funkci definující počáteční stav úložiště. V prvním parametru je jí předávána funkce `set()`. Ta slouží k aktualizaci stavu, přičemž přijímá objekt s vlastnostmi, jejichž hodnota se má aktualizovat. Funkce automaticky zajistí sloučení předaného objektu se stávajícím stavem úložiště a notifikuje o změně pouze relevantní React komponenty. Funkce `create()` spolu s úložištěm vytváří i hook, který se používá pro přístup k datům úložiště v React komponentách. Danému hooku lze předat selektor zajišťující odběr pouze specifických částí úložiště. Příklad úložiště `useNewProjectStore` přímo z aplikace je možné vidět v ukázce 2 – obsahuje dvě vlastnosti a dvě akce pro jejich modifikaci.

Aplikace obsahuje několik úložišť pro jasné oddělení zodpovědností. Především tedy každá stránka aplikace má své vlastní úložiště. Větší úložiště jsou dále rozdělena na menší, znovupoužitelné části podle vzoru *Slices Pattern* [36], čímž se zvyšuje udržitelnost kódu.

```

1 import { create } from "zustand";
2 import { DialogState } from "../types/DialogState";
3
4 type NewProjectStore = {
5   selectedFile: File | null | undefined,
6   dialogState: DialogState | null,
7   setSelectedFile: (file: File | null | undefined) => void,
8   setDialogState: (dialogState: DialogState) => void,
9 }
10
11 const useNewProjectStore = create<NewProjectStore>((set) => ({
12   selectedFile: null,
13   dialogState: null,
14   setSelectedFile: (selectedFile) => set({ selectedFile }),
15   setDialogState: (dialogState) => set({ dialogState }),
16 }));

```

Zdrojový kód 2: Úložiště useNewProjectStore.

6.5.1 With-transformace

V aplikaci využívám vlastní konvenci tzv. *with-transformací*. Jedná se o funkce (pojmenované s předponou *with*), které přijímají navrhovaný nový stav a aktuální stav úložiště. Jejich účelem je dopočítat odvozené hodnoty nebo zajistit konzistenci nového stavu před jeho finálním uložením do úložiště. With-transformace se volají v akcích úložiště a jejich výsledek se předává funkci `set()`. Příklad konkrétní *with-transformace* je možné vidět v ukázce 3.

```

1 export default function withLayoutBox(
2   newState: Partial<ExplorerStore>,
3   oldState: ExplorerStore
4 ): Partial<ExplorerStore> {
5   const concepts = withFallback(
6     newState.concepts, oldState.concepts);
7   return {
8     ...newState,
9     layoutBox: calculateLayoutBox(concepts),
10   };
11 }

```

Zdrojový kód 3: With-transformace withLayoutBox.

6.6 Datová reprezentace

Základem pro efektivní výpočty je vhodná volba datových struktur. Vzhledem k potenciálně velkému rozsahu dat jsem při návrhu datových struktur kladl důraz na minimalizaci paměťové náročnosti a rychlost přístupu k jednotlivým prvkům.

6.6.1 Formální kontext

Formální kontext je v paměti uložen jako objekt, jehož TypeScript typ je uveden v ukázce 4. Pole `objects` a `attributes` uchovávají názvy objektů a atributů. Relace „objekt má atribut“ je reprezentována binární maticí, kde řádky odpovídají objektům a sloupce atributům. Tato matice je uložena ve formě bitového pole ve vlastnosti `relation`. Na každý řádek dané matice připadá sekvence `cellsPerObject` čísel.

Vlastnost `cellSize` určuje maximální počet bitů využitých v každém čísle bitového pole. Jelikož JavaScript při bitových operacích interně pracuje s 32bitovými celými čísly se znaménkem, je tato vlastnost vždy nastavena na 32 bitů. Stejná šířka datového typu (`unsigned int`) je pak využita i při zpracování v jazyce C++, což zjednodušuje mapování dat mezi oběma prostředími.

```
1 export type FormalContext = {
2   readonly name?: string,
3   readonly relation: ReadonlyArray<number>,
4   readonly cellsPerObject: number,
5   readonly cellSize: number,
6   readonly objects: ReadonlyArray<string>,
7   readonly attributes: ReadonlyArray<string>,
8 }
```

Zdrojový kód 4: Typ `FormalContext`.

6.6.2 Formální koncepty

Formální koncepty jsou uloženy jako pole objektů. Každý koncept nese svůj index v daném poli, který slouží jako jeho unikátní identifikátor a pomocí kterého se na daný koncept odkazuje v dalších strukturách. `Extent` a `intent` konceptu je uložen jako pole indexů odkazujících na objekty a atributy v objektu formálního kontextu. Odpovídající TypeScript typy jsou uvedeny v ukázce 5. V C++ je pro pole indexů použit typ `std::vector<int>`.

```
1 export type FormalConcepts = ReadonlyArray<FormalConcept>
2
3 export type FormalConcept = {
4   readonly index: number,
5   readonly objects: ReadonlyArray<number>,
6   readonly attributes: ReadonlyArray<number>,
7 }
```

Zdrojový kód 5: Typ `FormalConcept`.

6.6.3 Konceptuální svaz

Konceptuální svaz je v rámci aplikace reprezentován jako orientovaný acyklický graf, kde vrcholy tvoří jednotlivé koncepty a hrany odpovídají relaci pokrytí. Hrany jsou uloženy pomocí dvou polí sousedů (pro obě možnosti orientace hran):

- `subconceptsRelation` – prvky pole jsou množiny indexů sousedních podkonceptů,
- `superconceptsRelation` – prvky pole jsou množiny indexů sousedních nadkonceptů.

Index v poli přímo odpovídá unikátnímu identifikátoru daného konceptu. Tento způsob uložení umožňuje efektivní průchod svazu v obou směrech. Použití množin zajišťuje možnost ověření existence hrany mezi dvěma koncepty v konstantním čase.

Objektové a atributové značení (přiřazení vlastních objektů a atributů) je realizováno pomocí slovníků `objectsLabeling` a `attributesLabeling`. Klíčem je vždy identifikátor konceptu a hodnotou pole indexů odkazujících na objekty nebo atributy v objektu formálního kontextu.

V TypeScriptu je pro množinu indexů použit typ `Set<number>`, v C++ typ `std::unordered_set<int>`. Odpovídající TypeScript typy jsou uvedeny v ukázce 6.

```
1 export type ConceptLattice = {
2   readonly subconceptsRelation: Relation,
3   readonly superconceptsRelation: Relation,
4   readonly attributesLabeling: ConceptLatticeLabeling,
5   readonly objectsLabeling: ConceptLatticeLabeling,
6 }
7
8 export type Relation = ReadonlyArray<ReadonlySet<number>>
9
10 export type ConceptLatticeLabeling =
11   ReadonlyMap<number, ReadonlyArray<number>>
```

Zdrojový kód 6: Typ `ConceptLattice`.

6.6.4 Rozložení diagramu

Rozložení vrcholů Hasseova diagramu je uloženo jako pole objektů s vlastnostmi pro souřadnice v trojrozměrném prostoru a pro identifikátor konceptu. Odpovídající TypeScript typy jsou uvedeny v ukázce 7. V C++ se pracuje pouze se samotnými souřadnicemi, které jsou pro vyšší efektivitu rozbaleny do vektoru typu `float`.

```

1 export type ConceptLatticeLayout = ReadonlyArray<ConceptPoint>
2
3 export type ConceptPoint = {
4     x: number,
5     y: number,
6     z: number,
7     conceptIndex: number,
8 }

```

Zdrojový kód 7: Typ ConceptLatticeLayout.

6.7 Organizace kódu

Kód aplikace je rozdělen do funkcí, které jsou dále podle poskytované funkcionality rozděleny do souborů a ty následně do adresářů. Nejdůležitější adresáře a soubory jsou následující:

```

root/
├─ benchmarks/ .....zdrojové kódy a skripty benchmarků
├─ datasets/ .....datasety pro testování
├─ scripts/ .....pomocné skripty
├─ src/ .....všechny zdrojové kódy aplikace
│   ├── assets/ .....statický obsah
│   ├── components/ .....React komponenty
│   ├── constants/ .....TypeScript konstanty
│   ├── contexts/ .....React kontexty
│   ├── cpp/ .....C++ kód
│   ├── css/ .....CSS kód
│   ├── hooks/ .....React hooks
│   ├── services/ .....služby
│   ├── stores/ .....Zustand úložiště
│   ├── types/ .....TypeScript typy
│   ├── utils/ .....pomocné funkce
│   └─ workers/ .....skripty Web Workerů
├─ tests/ .....automatizované testy
├─ emscripten.build.sh .....skript pro kompilaci C++ kódu
├─ index.html .....základní HTML soubor
└─ package.json .....konfigurace a závislosti projektu

```

7 Implementace algoritmů

Pro dosažení co nejvyššího výpočetního výkonu v prostředí webového prohlížeče jsem se rozhodl použít technologii WebAssembly (WASM). Jedná se o instrukční sadu umožňující vykonávat kód téměř nativní rychlostí, což je pro výpočetně náročné algoritmy žádoucí.

Původně jsem pro implementaci algoritmů zvažoval jazyk AssemblyScript, tedy jazyk podobný TypeScriptu, který se kompiluje do WebAssembly. Ten jsem však po úvodních testech zamítnul, protože nedosahoval požadovaného výkonu. Pro výslednou implementaci všech algoritmů jsem zvolil jazyk C++. Kompilace do WebAssembly je realizována pomocí nástroje Emscripten [37].

7.1 C++ kód

Kód je postaven téměř výhradně na standardních knihovnách C++, což minimalizuje velikost výsledného WebAssembly souboru. Jedinou vnější závislostí je knihovna Eigen [38], která se využívá pro implementaci metody PCA v algoritmu ReDraw. Pro správu paměti jsem zvolil chytré ukazatele (smart pointers), abych snížil pravděpodobnost vzniku úniků paměti.

Samotný kód je členěný do funkcí. Ty, které je potřeba volat z prostředí JavaScriptu, jsou exportovány spolu s příslušnými datovými strukturami pomocí nástroje Embind. Ten umožňuje mapovat C++ entity do JavaScriptového modulu, díky čemuž lze s funkcemi i daty pracovat jako s běžnými objekty. To značně zjednodušuje předávání dat mezi prostředím JavaScriptu a WebAssembly.

Jelikož se jedná o mou první zkušenost s jazykem C++, zaměřil jsem se především na správnost implementace algoritmů. Je pravděpodobné, že kód není z hlediska správy prostředků plně optimalizován pro maximální možný výkon.

7.2 Benchmark implementací algoritmu In-Close

Aby byla volba C++ podložena daty, provedl jsem benchmark implementací algoritmu In-Close ve třech variantách:

- C++ – Kód je kompilovaný do WebAssembly (Emscripten) a pro účely měření také do nativního kódu (Clang).
- AssemblyScript – Kód je kompilovaný do WebAssembly.
- TypeScript – Kód je transpilovaný do JavaScriptu a spouštěn v příslušných běhových prostředích.

Při kompilaci jsem u všech kompilátorů aktivoval optimalizace pro dosažení maximálního výkonu (například přepínač `-O3`).

Výsledné časy v tabulce 2 představují aritmetický průměr z 50 po sobě jdoucích měření. Jako testovací dataset jsem použil dataset *Mushroom* [39]. Ten obsahuje 8 124 objektů, 126 atributů a 233 116 konceptů. Všechny skripty benchmarku

jsou spolu s použitým datasetem přiloženy v elektronických datech. Měření jsem provedl na dvou odlišných platformách:

- macOS – procesor Apple M1, 16 GB RAM,
- Windows – procesor Intel Core i5-1135G7, 16 GB RAM.

Implementace	Prostředí	macOS (ms)	Windows (ms)
C++	Nativní (Clang)	727,70	1080,68
C++	Node.js	876,92	1044,26
C++	Google Chrome	904,48	1164,12
C++	Firefox	924,48	1252,76
C++	Safari	924,94	—
AssemblyScript	Node.js	3626,58	4586,70
AssemblyScript	Google Chrome	3024,68	3878,84
AssemblyScript	Firefox	2102,68	2359,20
AssemblyScript	Safari	1541,74	—
TypeScript	Node.js	8729,52	10138,06
TypeScript	Google Chrome	1855,84	2446,82
TypeScript	Firefox	5138,22	7829,22
TypeScript	Safari	1315,36	—

Tabulka 2: Srovnání implementací algoritmu In-Close v různých prostředích.

Z naměřených dat vyplývá, že C++ varianta nabízí nejvyšší výkon s poměrně nízkou režii při běhu ve WebAssembly. Výkon C++ varianty je v porovnání s ostatními stabilnější a předvídatelnější napříč všemi prostředími. Výsledky zároveň potvrzují nedostatečný výkon AssemblyScriptu, kde je doba výpočtu oproti C++ variantě v některých případech až trojnásobná. Zajímavým zjištěním je vysoká efektivita interpretace JavaScript kódu v prohlížečích Safari a Google Chrome, která však stále zaostává za WebAssembly kódem generovaným z C++.

8 Vizualizace rozsáhlých dat

Aktuální verze nástroje umožňuje vizualizaci konceptuálních svazů v řádu nižších stovek tisíc konceptů, a to při zachování dostatečné stability a plynulosti uživatelského rozhraní. Velmi však záleží na velikosti samotných konceptů (tj. velikosti extentu a intentu). I relativně malý konceptuální svaz (tisíce konceptů) může totiž vyžadovat nadměrné množství paměti, pokud jsou jeho koncepty rozsáhlé. Limitem je zde zejména nástroj Emscripten, který standardně omezuje velikost adresovatelné paměti na 2 GB.

V předchozích kapitolách již bylo zmíněno několik technik a technologií, které umožnily výpočet a efektivní uložení takto rozsáhlých svazů, konkrétně tedy Web Workers API, zvolená datová reprezentace, WebAssembly a jazyk C++. Tato kapitola se zaměřuje na jejich samotné zobrazení. V nástroji lze data konceptuálního svazu zobrazit dvěma způsoby:

- zobrazení formálního kontextu a konceptů formou seznamů a tabulek,
- vykreslení Hasseova diagramu, případně jeho části.

8.1 Virtualizace a líné načítání

Mezi hlavní faktory ovlivňující rychlost vykreslování obsahu ve webových prohlížečích patří množství elementů v DOM. Pro omezení velikosti DOM jsem proto při vykreslování dlouhých seznamů a tabulek implementoval dvě techniky:

- Líné načítání – Elementy seznamu se do DOM přidávají až ve chvíli, kdy se uživatel posune na aktuální konec daného seznamu.
- Virtualizace – Jedná se o rozšíření líného načítání. Elementy se do DOM přidávají až ve chvíli, kdy by se měly nacházet ve viditelné části obrazovky (viewportu), a zároveň se odebírají ty elementy, které již nejsou viditelné.

Jednoduché líné načítání je použito k zobrazení prostých seznamů objektů, atributů a konceptů. Tato volba vychází z předpokladu, že uživatelé nebudou manuálně procházet seznam tisíců položek, které by bylo nutné všechny přidat do DOM, ale raději využijí funkcionalitu vyhledávání nebo filtrování. U těchto seznamů by navíc implementace virtualizace byla příliš složitá, protože výška elementů nemusí být pro všechny položky stejná.

Virtualizace je použita především pro tabulku formálního kontextu. Zvolil jsem implementaci využívající CSS grid. Jednotlivá políčka tabulky se vkládají do mřížky, která má nastavené pevně dané rozměry a počty řádků a sloupců – políčka jsou čtverce konstatní velikosti. Díky tomu, že jsou rozměry tabulky a všech jejích políček vždy známé, lze z aktuálního posunu tabulky (scroll offset) snadno určit, jaká políčka mají být viditelná a vložit do DOM pouze ta.

Dále je virtualizace použita i pro zobrazení náhledu souboru v dialogích pro export dat. Řádkům mřížky odpovídají jednotlivé řádky souboru a sloupec má mřížka pouze jeden.

8.2 Možnosti vykreslování Hasseova diagramu

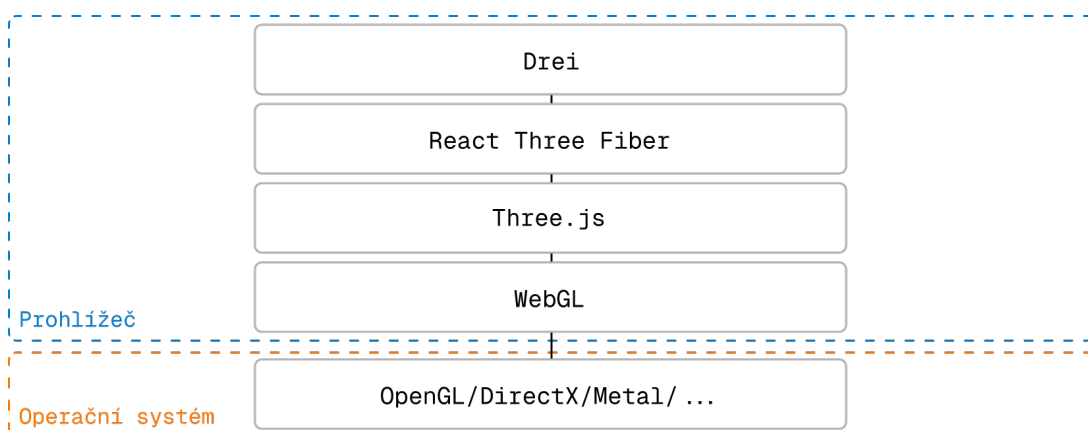
K vykreslení Hasseova diagramu ve webovém prohlížeči lze použít buď element `<svg>`, anebo element `<canvas>`. V případě elementu `<svg>` však u velkých svazů vyvstává dříve zmíněný problém velkého množství elementů v DOM. Proto jsem pro tento účel zvolil element `<canvas>`, a to ve dvou formách:

- Canvas API – Vysokoúrovňové API pro kreslení 2D grafiky na element `<canvas>`. Vyniká zejména svou jednoduchostí.
- WebGL API – API pro hardwarově akcelerované kreslení 2D a 3D grafiky na element `<canvas>`. Díky nízkoúrovňovému přístupu nabízí více možností optimalizace.

8.3 Interaktivní vizualizace Hasseova diagramu

WebGL API jsem se rozhodl využít pro implementaci interaktivní vizualizace Hasseova diagramu, která umožňuje jeho posouvání, přibližování, otáčení, výběr vrcholů a jejich přesouvání. Při použití Canvas API jsem u větších diagramů nebyl schopen dosáhnout požadované plynulosti uživatelského rozhraní.

S WebGL API nepracuji přímo, ale pomocí knihoven Three.js [40] a React Three Fiber [41]. Three.js odstiňuje vývojáře od složitých nízkoúrovňových detailů, jako jsou například maticové transformace, výpočty osvětlení nebo shadery, a umožňuje budovat komplexní scény pomocí objektového přístupu. React Three Fiber je pak nadstavbou nad knihovnou Three.js, která umožňuje celou scénu definovat deklarativním přístupem pomocí React komponent. Dále ještě používám knihovnu Drei [42] rozšiřující React Three Fiber o množství znovupoužitelných komponent (například pro vykreslování textu). Vztah mezi těmito technologiemi je znázorněn na obrázku 11.



Obrázek 11: WebGL, Three.js, React Three Fiber a Drei.

8.3.1 Geometrie a materiál

Objekty vykreslované scény jsou ve Three.js reprezentovány JavaScript objekty typu `Mesh`. Ty se skládají z geometrie a materiálu. Geometrie definuje povrch objektu, tedy souřadnice vrcholů a jejich propojení. Ve WebGL jsou povrchy objektů standardně tvořeny sítí trojúhelníků. Materiál pak definuje vizuální vlastnosti objektu, tedy jakou má mít například barvu nebo jak má odrážet světlo. Interně materiál zapouzdřuje shadery, parametry shaderů a případně textury.

8.3.2 Omezení počtu trojúhelníků

Zásadním faktorem ovlivňujícím výkon je složitost vykreslovaných objektů, tedy celkový počet trojúhelníků, které tvoří jejich povrch. Je žádoucí tento počet co nejvíce minimalizovat, a proto jsem pro vizualizaci zvolil geometrie s minimální nutnou podrobností:

- Vrcholy – Jsou reprezentovány geometrií koule (`SphereGeometry`) se záměrně sníženým počtem trojúhelníků. Přestože geometrie není při blízkém pohledu dokonale hladká, pro účely vizualizace diagramu je plně postačující a napomáhá plynulejšímu vykreslování i při velkém počtu vrcholů.
- Hrany – V závislosti na režimu zobrazení (2D nebo 3D) se využívají buď obdélníky (`ShapeGeometry`), anebo trojboké hranoly (`TubeGeometry`). I ve 3D režimu zobrazení má uživatel navíc možnost přepnout zobrazení hran na obdélníky pro další úsporu.

K lepšímu výkonu taktéž přispívá optimalizace zvaná *back-face culling*, která je ve Three.js ve výchozím stavu aktivována. Zajišťuje, že se vykreslují pouze vnější stěny trojúhelníků objektu. Vnitřní stěny, které by uživatel za běžných okolností neviděl, jsou ignorovány. U obdélníkových hran se však ve 3D režimu zobrazení musí vykreslovat oba typy stěn.

8.3.3 Instanced rendering

Příkazy na vykreslení objektu (*draw calls*), vyvolané například změnou jeho pozice, velikosti nebo barvy, s sebou nesou jistou režii – CPU musí připravit stav GPU a odeslat příkaz. Přestože tato režie není vysoká, při velkém počtu objektů je znatelná a prakticky nedovoluje vykreslovat rozsáhlé Hasseovy diagramy za současného zachování přijatelné interaktivity. Toto však platí pouze za předpokladu, že je každý vrchol a každá hrana diagramu samostatným objektem. Každému objektu totiž odpovídá jeden příkaz na vykreslení.

WebGL podporuje techniku zvanou *instanced rendering*, která umožňuje efektivně vykreslovat velké množství vizuálně identických objektů, zde označovaných jako *instance*. Díky této technice je možné odeslat data identických objektů do paměti pouze jednou a následně v rámci jediného příkazu vykreslit několik instancí najednou. Atributy (transformace a barva) jednotlivých instancí se získávají z předpřipravených bufferů.

Ve Three.js je instanced rendering realizován objekty typu `InstancedMesh`, v React Three Fiber komponentou `<instancedMesh>`. Tuto komponentu využívám pro vykreslování vrcholů i hran. Příklad vlastní React Three Fiber komponenty vykreslující všechny hrany diagramu je možné vidět v ukázce 8. Nastavení barvy jednotlivých hran řeší hook `useLinksColor()` a nastavení transformace hook `useLinksTransformation()`.

```

1  export default function Links() {
2    const instancedMeshRef = useRef<InstancedMesh>(null);
3    const links = useLinks();
4    const { visible, flat, semitransparent } = useLinksState();
5    useLinksTransformation(instancedMeshRef, links);
6    useLinksColor(instancedMeshRef, links);
7
8    return (
9      <instancedMesh
10        ref={instancedMeshRef}
11        args={[undefined, undefined, links.length]}
12        visible={visible}
13        frustumCulled={false}>
14        {flat ?
15          <shapeGeometry
16            args={[FLAT_LINE_SHAPE]} /> :
17          <tubeGeometry
18            args={[TUBE_LINE_CURVE, 1, 0.5, 3, false]} />}
19        <meshBasicMaterial
20          transparent={semitransparent}
21          opacity={semitransparent ? 0.3 : 1}
22          side={flat ? DoubleSide : FrontSide} />
23      </instancedMesh>
24    );
25  }

```

Zdrojový kód 8: Vlastní React Three Fiber komponenta `Links`.

8.3.4 Další optimalizace

Nástroj dále poskytuje několik nastavení, jejichž vypnutí přispívá k lepšímu výkonu na úkor vizuální kvality nebo uživatelské přívětivosti. Konkrétně se jedná o možnost vypnout:

- Antialiasing – Vyhlazování hran objektů s sebou nese dodatečnou režii.
- Barevné zvýrazňování hran při označení vrcholu – Aktualizace barvy hran vyžaduje výpočty na CPU a následný zápis do paměti GPU.
- Průhlednost hran – Vykreslování průhledných objektů je náročnější než vykreslování neprůhledných objektů.

8.4 Exportovaná vizualizace Hasseova diagramu

Vizualizace diagramu určená k exportu do rastrových formátů je realizována pomocí Canvas API. To pro vykreslování 2D scén nabízí vyšší přesnost a intuitivnější implementaci než WebGL API. Přestože je překreslování scény u větších diagramů znatelně pomalejší, v tomto případě se nejedná o zásadní problém. Na rozdíl od interaktivní vizualizace zde totiž není vyžadováno překreslování v reálném čase. Scéna se ve finálním rozlišení vykresluje pouze při změně parametrů exportu.

Pro zachování co nejvyšší plynulosti uživatelského rozhraní jsem využil objekt `OffscreenCanvas`. Ten ve spojení s Web Workers API umožňuje přenést proces vykreslování do pracovního vlákna, čímž se zamezuje blokování hlavního vlákna. V pracovním vlákne zároveň probíhá i samotné generování výsledného rastrového souboru.

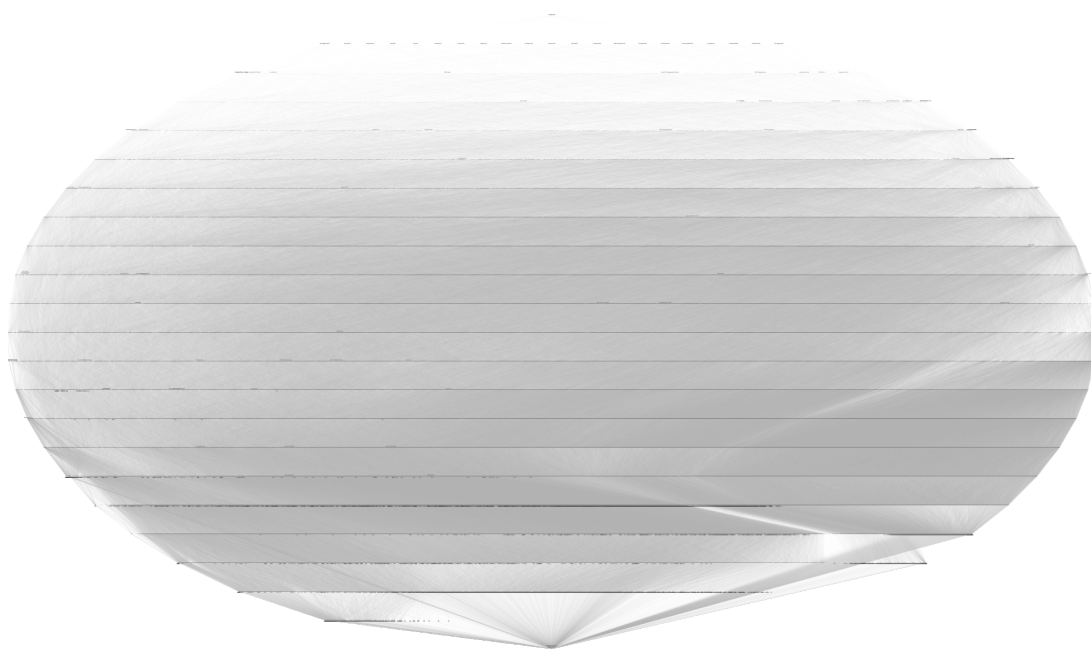
Všechny webové prohlížeče mají zavedeno omezení na rozměry plátna elementu `<canvas>` (respektive objektu `OffscreenCanvas`). Proto jsou i v mém nástroji rozměry exportovaných obrázků omezeny. Šířka a výška obrázku je omezena na nejvýše 32 767 pixelů a celková plocha na nejvýše $16\,384 \times 16\,384$ pixelů. Tyto hodnoty jsem zvolil s ohledem na zajištění maximální kompatibility napříč webovými prohlížeči [43].

8.5 Limity nástroje

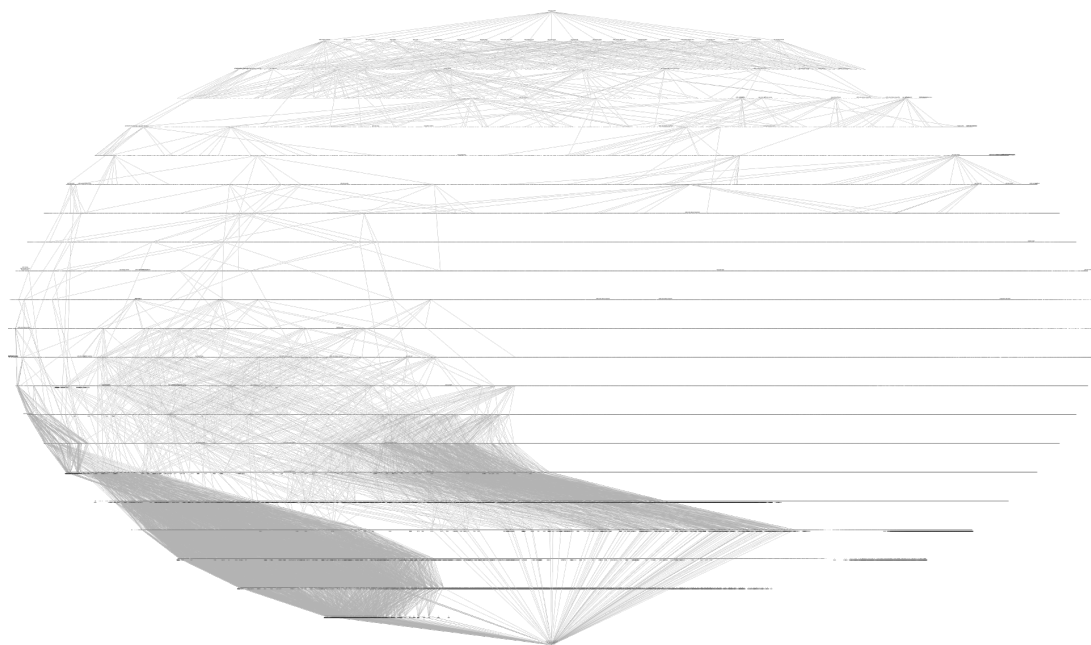
Největší konceptuální svaz, který jsem pomocí nástroje vizualizoval, je konceptuální svaz datasetu *Mushroom*. Hasseův diagram tohoto svazu tvoří celkem 233 116 vrcholů a 1 339 713 hran. Pro výpočet rozložení vrcholů diagramu jsem zvolil Sugiyama framework s horizontálním rozmístěním do elipsy. Diagram se mi povedlo exportovat ve formátu PNG v rozlišení $14\,043 \times 8\,184$ pixelů. Zmenšený výsledek lze vidět na obrázku 12.

Na platformě macOS (procesor Apple M1, 16 GB RAM) je nástroj i při zpracovávání takto rozsáhlého datasetu dostatečně stabilní a plynulý ve všech standardních webových prohlížečích – Google Chrome, Firefox a Safari. Jako nejstabilnější se ukázal prohlížeč Google Chrome. Největší rozdíly v chování prohlížečů jsem zaznamenal u funkcionality exportu diagramu do rastrového formátu. Prohlížeč Safari pomocí Canvas API nedokáže takto velké diagramy kompletně vykreslit a ve výsledku například chybí některé hrany (k vidění na obrázku 13). Prohlížeč Firefox je pak ve vykreslování pomocí Canvas API znatelně pomalejší.

Plynulost interaktivní vizualizace zásadně ovlivňuje zobrazení popisků vrcholů. V prohlížeči Google Chrome dosahuje snímková frekvence při manipulaci s diagramem datasetu *Mushroom* bez zobrazených popisků přibližně 46 FPS. Spotřeba operační paměti se pohybuje okolo 1,2 GB. Při povolení popisků snímková frekvence ale klesá až k 7 FPS a využití paměti stoupá k 1,5 GB. K výrazným propadům výkonu dochází také při interaktivních operacích, jako je zvýraznění incidentních hran vybraného vrcholu. Snímkovou frekvenci a spotřebu paměti jsem měřil pomocí vývojářských nástrojů a správce úloh prohlížeče.



Obrázek 12: Hasseův diagram konceptuálního svazu datasetu *Mushroom* vykreslený pomocí Canvas API v prohlížeči Google Chrome.



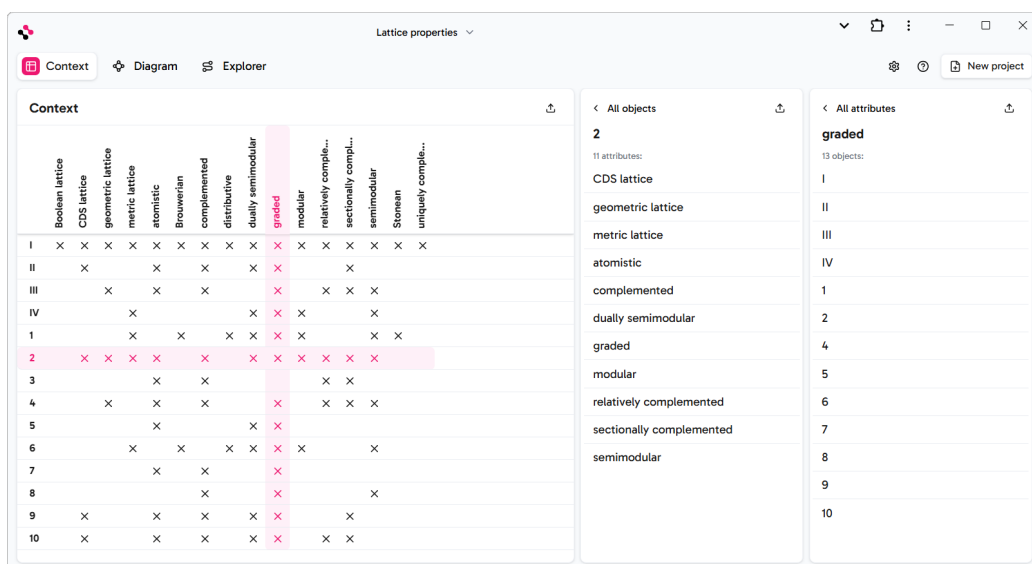
Obrázek 13: Hasseův diagram konceptuálního svazu datasetu *Mushroom* vykreslený pomocí Canvas API v prohlížeči Safari.

9 Možnosti nástroje

Nástroj konlatt je určen především pro desktopové prohlížeče, ve kterých poskytuje nejlepší uživatelský zážitek. Uživatelské rozhraní je sice responzivní, na mobilních zařízeních však jeho používání není ideální. Díky podpoře funkcionality PWA lze s nástrojem pracovat podobně jako s nativní desktopovou aplikací.

9.1 Nahrání a zobrazení formálního kontextu

Úvodní stránka nástroje obsahuje zejména vstupní pole pro nahrání analyzovaného datasetu ve formátu Burmeister, CSV, JSON nebo XML. Informace o podporovaných vstupních i výstupních formátech včetně příkladů jsou dostupné přímo v nástroji. Jakmile proběhne nahrání dat, je uživatel přesměrován na stránku s tabulkou formálního kontextu (obrázek 14). Tabulka umožňuje vyhledávání konkrétních objektů nebo atributů a základní filtraci. Formální kontext lze exportovat, a to včetně samostatných seznamů objektů a atributů.

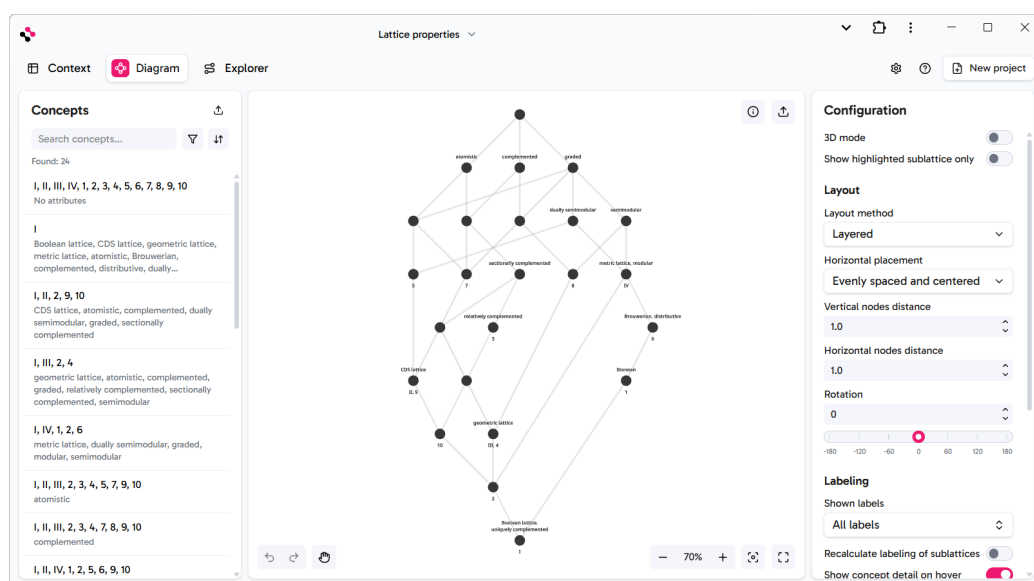


	Boolean lattice	CDS lattice	geometric lattice	metric lattice	atomistic	Brouwerian	complemented	distributive	dually semimodular	graded	modular	relatively comple...	sectionally compl...	semimodular	Stonean	uniquely comple...
I	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
II		x				x	x	x	x	x		x	x			
III			x	x	x		x		x	x		x	x	x		
IV				x					x	x				x		
1				x		x		x	x	x		x	x		x	x
2		x	x	x	x		x		x	x	x	x	x	x		
3				x	x							x	x			
4			x	x	x					x		x	x	x		
5				x					x	x						
6				x	x		x	x	x	x		x		x		
7					x		x			x						
8							x			x					x	
9		x		x	x		x	x	x	x			x			
10		x		x	x		x	x	x	x		x	x			

Obrázek 14: Tabulka formálního kontextu v nástroji konlatt.

9.2 Interaktivní vizualizace konceptuálního svazu

Stěžejní část aplikace tvoří stránka s interaktivní vizualizací konceptuálního svazu formou Hasseova diagramu (obrázek 15). Diagram využívá redukované objektové a atributové značení, přičemž kompletní detail konceptu se zobrazí po najetí na odpovídající vrchol. Rozložení vrcholů diagramu se generuje automaticky podle metody zvolené v konfiguračním panelu. Panel dále slouží k nastavení režimu zobrazení diagramu (2D nebo 3D), úpravě horizontálních a vertikálních rozestupů, rotaci diagramu nebo změně viditelnosti popisků a zvýraznění hran.



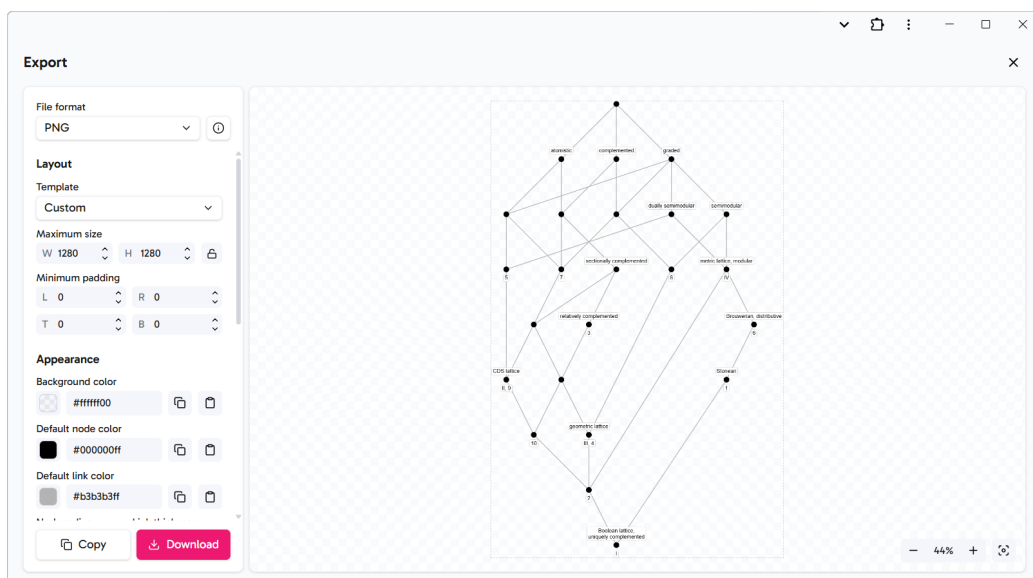
Obrázek 15: Interaktivní vizualizace konceptuálního svazu v nástroji konlatt.

Uživatel může diagram přibližovat, posouvat a otáčet. Taktéž může libovolně manuálně přemísťovat vrcholy daného diagramu, a to jednotlivě i hromadně. Veškeré manuální úpravy rozložení podporují historii změn (undo/redo). Pro vyšší přehlednost při práci s rozsáhlými konceptuálními svazy nástroj umožňuje přepnutí vizualizace do režimu celé obrazovky.

Vedle samotné vizualizace stránka obsahuje panel se seznamem všech konceptů, který podporuje vyhledávání a filtraci. Celý seznam konceptů je možné exportovat, a to včetně relace pokrytí samotného konceptuálního svazu. Kliknutí na vrchol diagramu nebo výběr ze seznamu konceptů zobrazí detail s příslušnými objekty a atributy. Uživatel má také možnost zvýraznit dolní nebo horní kužel konkrétního konceptu, čímž lze zobrazit určitý podsvaz konceptuálního svazu. Stejným způsobem nástroj umožňuje zvýraznit pouze průnik horního kužele jednoho konceptu a dolního kužele druhého.

9.3 Export vizualizace konceptuálního svazu

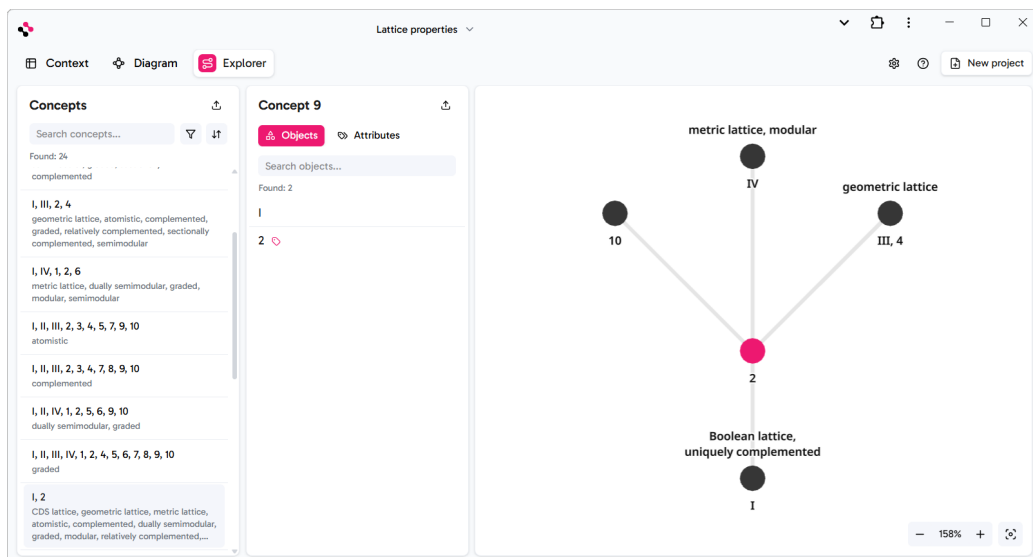
Hotový Hasseův diagram má uživatel možnost exportovat v několika formátech prostřednictvím modálního dialogu (obrázek 16). Kromě běžných rastrových formátů PNG a JPEG nástroj podporuje také vektorové formáty SVG a TikZ. Poslední zmíněný formát umožňuje vložit diagram přímo do $\text{\LaTeX}$ dokumentů, a zajistit tak co nejlepší kvalitu vykreslení při tisku. U formátů PNG, JPEG a SVG nástroj nabízí rozsáhlé možnosti přizpůsobení výsledného obrázku. Je možné nastavit rozměry, odsazení, barvu pozadí, vrcholů a hran nebo podobu popisků.



Obrázek 16: Export Hasseova diagramu konceptuálního svazu v nástroji konlatt.

9.4 Průzkumník sousedství

Detailnější pohled na konkrétní části konceptuálního svazu nabízí stránka s průzkumníkem sousedství (obrázek 17). Ten uživateli dovoluje se zaměřit na vybraný koncept a jeho bezprostřední okolí, tedy přímé nadkoncepty a podkoncepty. Kliknutím na libovolný sousední koncept se zobrazení aktualizuje pro daný koncept, což dovoluje navigaci celým konceptuálním svazem. Tento průzkumník je užitečný zejména u rozsáhlejších svazů, kde bývá celkový diagram nepřehledný.



Obrázek 17: Průzkumník sousedství v nástroji konlatt.

Závěr

Výsledkem této práce je nový webový nástroj pro vizualizaci a analýzu konceptuálních svazů. Hlavním cílem při jeho návrhu bylo odstranit nedostatky stávajících řešení a učinit formální konceptuální analýzu přístupnější širší škále uživatelů, a to i těm, kteří nepatří mezi úzkou skupinu technicky zdatných expertů v oboru.

V rámci implementace nástroje jsem kladl důraz na moderní, intuitivní a přehledné uživatelské rozhraní. Klíčovou funkcionalitou je interaktivní vizualizace konceptuálního svazu formou Hasseova diagramu, která umožňuje snadnou interpretaci dat. V nástroji jsem implementoval několik metod pro automatické kreslení Hasseova diagramu. Pro možnost dalšího využití diagramu nástroj podporuje jeho export v různých rastrových i vektorových formátech. Nástroj taktéž nabízí jednoduchý průzkumník sousedství formálních konceptů, který usnadňuje orientaci v rozsáhlých konceptuálních svazech.

Významnou část vývoje jsem věnoval optimalizaci výkonu. Nástroj tak umožňuje pracovat s konceptuálními svazy až v řádu nižších stovek tisíc konceptů, a to při zachování dostatečné stability a plynulosti uživatelského rozhraní. Tím překonává limity mnoha dosud dostupných nástrojů.

Conclusions

The result of this thesis is a new web tool for visualization and analysis of concept lattices. The main goal was to eliminate the shortcomings of existing solutions and make formal concept analysis more accessible to a wider range of users, even those who do not belong to a narrow group of technically proficient experts in the field.

During the implementation of the tool, I focused on the creation of a modern, intuitive and clear user interface. The key functionality is an interactive visualization of a concept lattice in the form of a Hasse diagram, which allows easy interpretation of data. I implemented several methods for the automatic layout of Hasse diagram nodes. To enable further use of the diagram, the tool supports its export in various raster and vector formats. The tool also features a simple neighborhood explorer of formal concepts, which simplifies orientation in large concept lattices.

I dedicated a significant part of the development to performance optimization. Consequently, the tool allows working with concept lattices of up to hundreds of thousands of concepts while maintaining sufficient stability and user interface responsiveness. In this regard, the tool overcomes the limits of many currently available tools.

A Zprovozněná aplikace

Nástroj je zprovozněn na <https://radekvym.github.io/konlatt>, případně je možné jej zprovoznit z přiložených zdrojových kódů.

B Obsah elektronických dat

text/

Adresář obsahující text diplomové práce ve formátu PDF a zdrojové kódy tohoto textu.

konlatt/

Adresář obsahující zdrojové kódy nástroje.

README.md

Textový soubor obsahující postup zprovoznění nástroje.

Literatura

- [1] Ganter, Bernhard; Wille, Rudolf. *Formal Concept Analysis - Mathematical Foundations*. 2024. Dostupný také z: <https://doi.org/10.1007/978-3-031-63422-2>. ISBN 978-3-031-63422-2.
- [2] Yevtushenko, S. A. System of data analysis Concept Explorer. In. *Proceedings of the 7th National Conference on Artificial Intelligence (KII-2000)*. 2000, s. 127–134. In Russian. Dostupný také z: <https://conexp.sourceforge.net/ConExp-2000.pdf>.
- [3] Vargas-García, Edith; Wachtel, Andreas. FCA using the Concept Explorer in 2024. *CoRR*. 2024, roč. abs/2411.06675. Dostupný také z: <https://doi.org/10.48550/arXiv.2411.06675>.
- [4] Becker, Peter; Correia, Joachim. The ToscanaJ Suite for Implementing Conceptual Information Systems. In. *The ToscanaJ Suite for Implementing Conceptual Information Systems*. 2005, s. 324–348. Dostupný také z: http://dx.doi.org/10.1007/11528784_17. ISBN 978-3-540-27891-7.
- [5] Valtchev, Petko; Roume, Cyril. Galicia: An Open Platform for Lattices. 2003.
- [6] Alam, Mehwish; Le, Thi Nhu Nguyen; Napoli, Amedeo. LatViz: A New Practical Tool for Performing Interactive Exploration over Concept Lattices. In. *CLA*. 2016, s. 9–20. CEUR Workshop Proceedings. Dostupný také z: <https://ceur-ws.org/Vol-1624/paper1.pdf>.
- [7] *FCA Tools Bundle*. [online]. [cit. 2026-2-26]. Dostupný z: <https://fca-tools-bundle.com>.
- [8] Demko, Christophe; Boukhetta, Salah Eddine; Richard, Jérémy aj. GALACTIC: towards a generic and scalable platform for complex and heterogeneous data using Formal Concept Analysis. In. *CLA*. 2022, s. 191–198. CEUR Workshop Proceedings. Dostupný také z: <https://ceur-ws.org/Vol-3308/Paper15.pdf>.
- [9] *Concepts*. [online]. [cit. 2026-2-26]. Dostupný z: <https://concepts.readthedocs.io>.
- [10] Hanika, Tom; Hirth, Johannes. Conexp-Clj - A Research Tool for FCA. In. *IC-FCA (Supplements)*. 2019, s. 70–75. CEUR Workshop Proceedings. Dostupný také z: <https://ceur-ws.org/Vol-2378/shortAT8.pdf>.
- [11] Cordero, Pablo; Enciso, Manuel; López-Rodríguez, Domingo; Mora, Ángel. fcaR, Formal Concept Analysis with R. *The R Journal*. 2022, roč. 14, s. 341–361. <https://doi.org/10.32614/RJ-2022-014>. Dostupný také z: <http://dx.doi.org/10.32614/RJ-2022-014>. ISSN 2073-4859.
- [12] Andrews, Simon. In-Close, a fast algorithm for computing formal concepts. 2009, roč. 483.
- [13] Carpineto, Claudio; Romano, Giovanni. *Concept data analysis - theory and applications*. 2005.

- [14] Battista, Giuseppe Di; Eades, Peter; Tamassia, Roberto; Tollis, Ioannis G. *Graph Drawing: Algorithms for the Visualization of Graphs*. 1999. ISBN 0-13-301615-3.
- [15] Sugiyama, Kozo; Tagawa, Shojiro; Toda, Mitsuhiro. Methods for Visual Understanding of Hierarchical System Structures. *IEEE Trans. Syst. Man Cybern.* 1981, roč. 11, č. 2, s. 109–125. Dostupný také z: <https://doi.org/10.1109/TSMC.1981.4308636>.
- [16] Jünger, Michael; Mutzel, Petra. 2-Layer Straightline Crossing Minimization: Performance of Exact and Heuristic Algorithms. *J. Graph Algorithms Appl.* 1997, roč. 1, č. 1, s. 1–25. Dostupný také z: <https://doi.org/10.7155/jgaa.00001>.
- [17] Barth, Wilhelm; Mutzel, Petra; Jünger, Michael. Simple and Efficient Bilayer Cross Counting. *J. Graph Algorithms Appl.* 2004, roč. 8, č. 2, s. 179–194. Dostupný také z: <https://doi.org/10.7155/jgaa.00088>.
- [18] Brandes, Ulrik; Köpf, Boris. Fast and Simple Horizontal Coordinate Assignment. In. *GD*. 2001, s. 31–44. Lecture Notes in Computer Science. Dostupný také z: https://doi.org/10.1007/3-540-45848-4_3.
- [19] *Formal Concept Analysis Examples*. [online]. [cit. 2026-3-9]. Dostupný z: <https://upriss.github.io/fca/examples.html>.
- [20] Freese, Ralph. Automated Lattice Drawing. In. *ICFCA*. 2004, s. 112–127. Lecture Notes in Computer Science. Dostupný také z: https://doi.org/10.1007/978-3-540-24651-0_12.
- [21] *Lattice Drawing*. [online]. [cit. 2026-3-10]. Dostupný z: <https://latdraw.org>.
- [22] Dürreschnabel, Dominik; Stumme, Gerd. Force-Directed Layout of Order Diagrams Using Dimensional Reduction. In. *ICFCA*. 2021, s. 224–240. Lecture Notes in Computer Science. Dostupný také z: https://doi.org/10.1007/978-3-030-77867-5_14.
- [23] *domduerr/redraw*. [online]. [cit. 2026-3-10]. Dostupný z: <https://github.com/domduerr/redraw>.
- [24] *React*. [online]. [cit. 2026-3-9]. Dostupný z: <https://react.dev>.
- [25] *Vite*. [online]. [cit. 2026-3-9]. Dostupný z: <https://vite.dev>.
- [26] *Vitest*. [online]. [cit. 2026-3-9]. Dostupný z: <https://vitest.dev>.
- [27] *Progressive web apps / MDN*. [online]. [cit. 2026-3-9]. Dostupný z: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps.
- [28] *Vite PWA*. [online]. [cit. 2026-3-9]. Dostupný z: <https://vite-pwa-org.netlify.app>.
- [29] *React Router Official Documentation*. [online]. [cit. 2026-3-9]. Dostupný z: <https://reactrouter.com>.

- [30] *Tailwind CSS*. [online]. [cit. 2026-3-9]. Dostupný z: [⟨https://tailwindcss.com⟩](https://tailwindcss.com).
- [31] *tailwind-merge - npm*. [online]. [cit. 2026-3-9]. Dostupný z: [⟨https://www.npmjs.com/package/tailwind-merge⟩](https://www.npmjs.com/package/tailwind-merge).
- [32] *clsx - npm*. [online]. [cit. 2026-3-9]. Dostupný z: [⟨https://www.npmjs.com/package/clsx⟩](https://www.npmjs.com/package/clsx).
- [33] *React Icons*. [online]. [cit. 2026-3-9]. Dostupný z: [⟨https://react-icons.github.io/react-icons⟩](https://react-icons.github.io/react-icons).
- [34] *fast-xml-parser - npm*. [online]. [cit. 2026-3-9]. Dostupný z: [⟨https://www.npmjs.com/package/fast-xml-parser⟩](https://www.npmjs.com/package/fast-xml-parser).
- [35] *Zustand*. [online]. [cit. 2026-3-5]. Dostupný z: [⟨https://zustand-demo.pmnd.rs⟩](https://zustand-demo.pmnd.rs).
- [36] *Slices Pattern - Zustand*. [online]. [cit. 2026-3-5]. Dostupný z: [⟨https://zustand.docs.pmnd.rs/learn/guides/slices-pattern⟩](https://zustand.docs.pmnd.rs/learn/guides/slices-pattern).
- [37] *Main — Emscripten documentation*. [online]. [cit. 2026-3-15]. Dostupný z: [⟨https://emscripten.org⟩](https://emscripten.org).
- [38] *Eigen: A C++ template library for linear algebra*. [online]. [cit. 2026-3-10]. Dostupný z: [⟨https://libeigen.gitlab.io⟩](https://libeigen.gitlab.io).
- [39] *Mushroom - UCI Machine Learning Repository*. [online]. 1981 [cit. 2026-4-28]. Dostupný z: [⟨https://doi.org/10.24432/C5959T⟩](https://doi.org/10.24432/C5959T).
- [40] *Three.js – JavaScript 3D Library*. [online]. [cit. 2026-3-15]. Dostupný z: [⟨https://threejs.org⟩](https://threejs.org).
- [41] *Introduction - React Three Fiber*. [online]. [cit. 2026-3-15]. Dostupný z: [⟨https://r3f.docs.pmnd.rs⟩](https://r3f.docs.pmnd.rs).
- [42] *Introduction - Drei*. [online]. [cit. 2026-3-15]. Dostupný z: [⟨https://drei.docs.pmnd.rs⟩](https://drei.docs.pmnd.rs).
- [43] *canvas-size*. [online]. [cit. 2026-3-22]. Dostupný z: [⟨https://jhildenbiddle.github.io/canvas-size/#/?id=test-results⟩](https://jhildenbiddle.github.io/canvas-size/#/?id=test-results).